

ANDREA DI ROCCO
GAMESHELL



ADR...



Introduzione al progetto GameShell

Andrea Di Rocco

In questo progetto mi sono apprestato a lavorare con **GameShell**, un ambiente simulato pensato per imparare ad usare il terminale Linux.

L'obiettivo principale era quello di completare tutte le **42 missioni** disponibili, che vanno da comandi base come spostarsi tra cartelle, fino ad arrivare a scrivere **veri e propri script personalizzati**.

Ho deciso di affrontarlo in maniera ordinata, partendo dalla prima missione e cercando di capire bene ogni comando, senza saltare nulla.

Non mi sono limitato a copiare, ma ho voluto **capire ogni passaggio**, testarlo, e anche sbagliare qualche volta per poi correggermi.

Man mano che andavo avanti, mi rendevo conto che non stavo solo imparando a scrivere comandi, ma stavo cominciando a **ragionare come si ragiona nella shell**: in modo logico, preciso, e soprattutto funzionale.

Ogni missione mi ha insegnato qualcosa di diverso, e alcune mi hanno anche fatto riflettere su come organizzare meglio il lavoro o su come automatizzare alcune operazioni.

GameShell mi ha dato la possibilità di imparare Linux in modo pratico, e questa è la cosa che più ho apprezzato.

Non si tratta solo di teoria, ma di mettersi lì, riga per riga, e **fare**. Ed è proprio quello che mi sono impegnato a fare, fino alla fine.

Avvio dell'ambiente e gioco

Dopo aver capito a cosa serviva questo esercizio, ho iniziato facendo le cose nel modo più pulito possibile.

Ho aperto Kali Linux nella mia macchina virtuale, e prima di iniziare ho pensato: "meglio se aggiorno tutto, così non ho problemi durante le missioni".

Sono andato sul terminale e ho lanciato questo comando per aggiornare il sistema:

```
sudo apt update && sudo apt upgrade
```

Ho aspettato che finisse tutto l'aggiornamento. Lo faccio sempre prima di iniziare qualcosa di nuovo, perché a volte anche un piccolo pacchetto non aggiornato può dare errori strani, e preferisco non rischiare.

Una volta completato tutto, ho chiuso e riaperto il terminale così da essere sicuro che tutto fosse ok.

A quel punto mi sono posizionato nella cartella dove avevo GameShell, ho controllato che il file `gameshell.sh` ci fosse con:

```
ls
```

E quando l'ho visto, ho dato il via al progetto con:

```
gameshell.sh
```


Missione 1 – La cima della torre del Castello

Obiettivo: Salire fino in cima alla torre principale del Castello.

Appena entrato in GameShell, ho capito subito che dovevo iniziare a ragionare come se stessi davvero dentro una rete di cartelle. L'obiettivo era chiaro: scalare la torre, piano dopo piano.

Mi sono spostato dentro il Castello, poi nel Palazzo_principale, e da lì ho seguito la struttura interna fino alla cima.

```
cd Castello
cd Palazzo_principale
cd Primo_piano
cd Secondo_piano
cd Cima_della_torre
gsh check
```

```
[mission 1] $ ls
Castello Foresta Giardino Montagna Stalla

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Castello

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Primo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Primo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ cd Cima_della_torre

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 1] $ ls
```

Congratulazioni, la missione 1 é stata completata con successo!

[progress was saved in /home/kali/gameshell-save.sh]

```
Dispositivi
--+-----+--
| Usa il comando |
| $ gsh help    |
| per ottenere una lista dei comandi "gsh". |
--+-----+--
|
```

Primo livello completato ma credo di avere qualche problema nel gioco poiché dovevo salire sopra la torre ma purtroppo dopo il primo piano non mi dava nulla, ho cercato un po' ovunque ma solamente dal palazzo principale mi faceva accedere fin sopra la torre e figuramente mi sembra strano, ma esaminando le varie directory solo da lì potevo salire, però alla fine primo livello completato.

Missione 2 Cantina

L'obiettivo della missione era semplice: raggiungere la cantina del castello. Mi trovavo ancora all'interno della torre, così ho iniziato a tornare indietro utilizzando i comandi necessari per risalire la struttura delle directory. Sono tornato alla radice, dove ho potuto vedere chiaramente le opzioni disponibili: Castello, Foresta, Giardino, Montagna e Stalla.

Mi sono diretto nel Castello e da lì ho visualizzato le stanze disponibili. Tra queste c'era la Cantina, la destinazione indicata. Una volta entrato, ho lanciato il comando per verificare la posizione e ho ricevuto conferma: missione completata con successo.

Durante gli spostamenti ho anche svolto l'obiettivo secondario, testando i comandi `cd ..` e `cd -` per capirne le differenze. Il primo permette di salire di un livello nella struttura delle cartelle, mentre il secondo consente di tornare alla directory precedente in cui ci si trovava, utile per muoversi avanti e indietro tra due posizioni.

Missione 2 completata correttamente, con entrambi gli obiettivi raggiunti.

```
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd Cantna
bash: cd: Cantna: File o directory non esistente

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd Cantina

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
botte_di_mele Palazzo_principale Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ gsh check

Congratulazioni, la missione 2 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]

Dispositivi
+-----+
| Usa il comando |
| $ gsh help    |
| per ottenere una lista dei comandi "gsh". |
+-----+
```

Comandi usati

`cd ..`

`ls`

`cd Castello`

`ls`

`cd Cantina`

`ls`

`gsh check`

Missione 3 – Sala del trono

L'obiettivo era tornare alla posizione iniziale e poi raggiungere la **Sala del trono**, ma con una regola ben precisa: farlo usando **solo due comandi**.

All'inizio mi sono preso il tempo per esplorare e capire bene dove mi trovavo e dove volevo arrivare. Ho fatto qualche prova, anche più di due comandi, per orientarmi meglio nella struttura.

Una volta individuato il percorso corretto, ho eseguito la sequenza definitiva:

Il secondo comando mi ha portato esattamente nella **stanza del trono**, passando in un colpo solo attraverso le varie cartelle necessarie.

```
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ cd

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ cd Castello/Palazzo_principale/Sala_del_trono

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ gsh check

Congratulazioni, la missione 3 é stata completata con successo!

Documenti Palazzo_principale Torre_principale
Musica
Immagini
Voci
/.';|:~::~:::' ||
|| spc || :~::~:::' _.;_-'-_-_-
|| _-|| :~::~:::' _-!oo @!-_-_'-
\'. _-|| :~::~:-!( )oo @!( )@-_'-_|
_-'-;|:-'-'.&$@.& ( )$%- 'o.' \U ||
Re '>'-.!@%( )@'_@_%- '_-o _-' | ||
jgs || _-'-_.@-_'-_'-_'-_-o _-' | ||
|| =[' _-'-_\U/_-' o _-' | || | | | |
|| _-'-|=| |'| o _-' | ||
|| || |'|
|| || |'|
```

Per questa missione dovevo creare una cartella chiamata **Capanna** dentro la **Foresta**, e poi una seconda cartella chiamata **Cassa** all'interno della Capanna.

Mi trovavo nella Sala_del_trono, quindi ho dovuto prima uscire da lì risalendo di due livelli fino alla base del Castello, e poi un terzo livello per tornare alla posizione iniziale.

Da lì ho potuto accedere finalmente alla Foresta. Una volta dentro, ho creato la cartella Capanna, ci sono entrato e ho creato al suo interno Cassa.

```
~/Castello/Palazzo_principale/Sala_del_trono
[mission 4] $ cd ..

~/Castello/Palazzo_principale
[mission 4] $ cd ..

~/Castello
[mission 4] $ cd ..
Cantina Grande_sala Osservatorio

~
[mission 4] $ ls
Castello Foresta Giardino Montagna Stalla

~
[mission 4] $ cd Foresta

~/Foresta
[mission 4] $ mkdir Capanna

~/Foresta
[mission 4] $ cd Capanna

~/Foresta/Capanna
[mission 4] $ mkdir Cassa

~/Foresta/Capanna
[mission 4] $ gsh check

Congratulazioni, la missione 4 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Comandi usati in ordine

```
cd ..
cd ..
cd ..
ls
cd Foresta
mkdir Capanna
cd Capanna
mkdir Cassa
gsh check
```

Missione 5 – Pulizia in Cantina

L'obiettivo di questa missione era tornare nella **Cantina** e eliminare tutti i **ragni**, lasciando però in pace i **pipistrelli**, perché fanno parte dello stemma del castello e portano fortuna.

Mi trovavo ancora nella Foresta, all'interno della Capanna, quindi ho dovuto tornare indietro fino alla posizione iniziale, poi entrare nel Castello e da lì raggiungere la Cantina.

Una volta dentro ho controllato i file presenti, ho identificato quelli che erano ragni e li ho eliminati usando il comando corretto.

I pipistrelli non li ho toccati, come richiesto dalla missione.

Ho lanciato `gsh check` e la missione è stata completata correttamente.

```
~/Castello/Cantina
[mission 5] $ ls
botte_di_mele  pipistrello_1  pipistrello_2  ragno_1  ragno_2  ragno_3

~/Castello/Cantina
[mission 5] $ rm ragno_1 ragno_2 ragno_3

~/Castello/Cantina
[mission 5] $ ls
botte_di_mele  pipistrello_1  pipistrello_2

~/Castello/Cantina
[mission 5] $ gsh check

Congratulazioni, la missione 5 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]

+-----+
| Usa il comando |
| $ gsh help    |
| per ottenere una lista dei comandi "gsh". |
+-----+
| Esplora la rete |
+-----+
```

Comandi usati in ordine

cd ~

cd Castello/Cantina

ls

rm ragno_1 ragno_2 ragno_3

gsh check

ps quando sono stato la prima volta nella cantina non c'erano minimamente ragni e pipistrelli...

Missione 6 – Trasferimento delle Monete

L'obiettivo era raccogliere tutte le **monete** presenti nel **Giardino** davanti al Castello e trasferirle nella **Cassa** che si trova dentro la **Capanna** nella **Foresta**.

Mi trovavo nella Cantina, quindi ho lasciato il Castello e mi sono diretto verso il **Giardino**. Lì ho controllato i file presenti e ho individuato tutte le monete.

Una volta raccolte, le ho spostate direttamente nella cartella **Cassa**, che si trova nel percorso completo: ~/Foresta/Capanna/Cassa.

Ho fatto attenzione a usare il simbolo ~ per riferirmi alla cartella iniziale, come suggerito nella missione.

Dopo aver completato lo spostamento, ho lanciato il comando di verifica e la missione è stata

completata correttamente.

```
|| Comandi utili ||
|| ===== ||
|| mv FILE1 FILE2 ... FILEn DIRECTORY ||
|| Sposta i file nella cartella. ||
|| Nota: ``mv`` e una abbreviazione di "move". ||
|| ||
|| ~ ||
|| Il "~" simbolo é una abbreviazione per la cartella iniziale. ||
|| Esempio: ovunque tu sia, ``~/Taverna`` indica la cartella (o ||
|| file) "Taverna" nella cartella iniziale. ||
|| ||
|| /s\/_/ ||
|| { _/_/_/ } ||
|| ||
|| ~/Castello/Cantina ||
|| [mission 6] $ cd ~/Giardino ||
|| ||
|| ~/Giardino ||
|| [mission 6] $ ls ||
|| Giardino_fiorito Labirinto moneta_1 moneta_2 moneta_3 Rimessa ||
|| ||
|| ~/Giardino ||
|| [mission 6] $ mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa ||
|| ||
|| ~/Giardino ||
|| [mission 6] $ gsh check ||
|| Congratulazioni, la missione 6 é stata completata con successo! ||
```

Comandi usati in ordine

```
cd ~/Giardino
ls
mv moneta_1 moneta_2 moneta_3 ~/Foresta/Capanna/Cassa
gsh check
```

Missione 7 – Monete nascoste nel Giardino

Questa missione richiedeva di trovare tutte le **monete nascoste** nel **Giardino** e spostarle nella **Cassa** della **Capanna** nella **Foresta**.

Appena ho lanciato `ls -A`, sono comparsi tre file nascosti, tutti con nomi lunghi e complicati che iniziavano con un punto, quindi invisibili con un semplice `ls`.

Ho provato a spostare tutte le monete in un solo comando, ma per errore ho incluso per sbaglio anche il comando `mv` tra i file da spostare, e quindi ho ricevuto un errore. Dopo il tentativo fallito, ho corretto tutto e ho eseguito il comando giusto, riuscendo a spostare tutte le monete.

Una volta che le monete sono sparite dalla cartella, ho dato `gsh check` e la missione è stata completata con successo.

```

~/Giardino
[mission 7] $ ls -A
.16485_moneta_3 .16642_moneta_2 .51258_moneta_1 Giardino_fiorito Labirinto Rimessa

~/Giardino
[mission 7] $ mv .51258_moneta_1 .16642_moneta_2 mv .16485_moneta_3 ~/Foresta/Capanna/Cassa
mv: impossibile eseguire stat di 'mv': File o directory non esistente

~/Giardino
[mission 7] $ .16485_moneta_3 .16642_moneta_2 .51258_moneta_1 ~/Foresta/Capanna/Cassa
.16485_moneta_3: comando non trovato

~/Giardino
[mission 7] $ ls -A
Giardino_fiorito Labirinto Rimessa

~/Giardino
[mission 7] $ gsh check

Congratulazioni, la missione 7 è stata completata con successo!

```

ls -A

.51258_moneta_1 .16642_moneta_2 .16485_moneta_3 ~/Foresta/Capanna/Cassa

gsh check

Missione 8 – Ragni sulle ragnatele

L'obiettivo era rimuovere tutti i **ragni** presenti nella **Cantina**, facendo attenzione a **non disturbare i pipistrelli**, come già indicato in missioni precedenti.

Mi trovavo nel **Giardino**, quindi ho usato un comando diretto per entrare nella Cantina. Una volta dentro, ho controllato il contenuto della cartella: erano presenti molti file, tutti ben riconoscibili.

I file dei ragni avevano tutti la parola **ragno** nel nome, quindi ho sfruttato la **wildcard** ***ragno*** per eliminarli tutti in un colpo solo, senza doverli scrivere uno a uno.

I pipistrelli e altri file, come **botte_di_mele**, sono rimasti intatti.

Alla fine, con **gsh check**, ho ricevuto conferma: missione completata con successo.

```

~/Giardino
[mission 8] $ cd ~/Castello/Cantina

~/Castello/Cantina
[mission 8] $ ls
10158_ragno_5      1674_ragno_22    2217_ragno_47    29991_ragno_4    4617_ragno_29
10808_pipistrello_1 17446_ragno_43  22210_ragno_1    30166_ragno_13   5714_ragno_28
12505_ragno_39    17541_ragno_34  23641_ragno_16   307_pipistrello_5 5894_ragno_49
12637_ragno_36    17554_ragno_25  24477_ragno_45   31480_ragno_40   6633_ragno_18
14407_ragno_6     17610_ragno_7   24564_ragno_9    31562_ragno_3    6843_ragno_21
15585_ragno_23    17839_ragno_10  24585_pipistrello_3 32124_ragno_42   7219_ragno_30
15632_ragno_31    1800_ragno_48   2513_pipistrello_4 32299_ragno_37   7729_ragno_50
15747_ragno_20    18619_ragno_46  26297_ragno_38   32583_pipistrello_2 botte_di_mele
15828_ragno_27    18776_ragno_24  26609_ragno_26   3304_ragno_19
1594_ragno_14     21372_ragno_15  28156_ragno_12   3535_ragno_32
16506_ragno_41    21432_ragno_35  28840_ragno_2    4074_ragno_44
16657_ragno_8     21716_ragno_33  29395_ragno_11   4353_ragno_17

~/Castello/Cantina
[mission 8] $ rm *ragno*

~/Castello/Cantina
[mission 8] $ ls
10808_pipistrello_1 2513_pipistrello_4 32583_pipistrello_2
24585_pipistrello_3 307_pipistrello_5  botte_di_mele

~/Castello/Cantina
[mission 8] $ gsh check

Congratulazioni, la missione 8 è stata completata con successo!

```

Comandi usati in ordine

```
cd ~/Castello/Cantina
```

```
ls
```

```
rm *ragno*
```

```
ls
```

```
gsh check
```

Missione 9 – Ragni nascosti nella Cantina

Questa volta i **ragni si erano nascosti** usando file nascosti, quindi non visibili con un semplice `ls`. Per individuarli, ho usato `ls -A` e ho trovato molti file che iniziavano con il punto (`.`), mescolati tra ragni e pipistrelli.

Ho provato prima con `rm .ragno*`, ma ho ricevuto un errore: il pattern non corrispondeva ai nomi reali.

Controllando meglio, ho notato che i file seguivano il formato `.qualcosa_ragno_qualcosa`, quindi ho usato il pattern corretto:

```
rm .*_ragno_*
```

Con questo comando ho eliminato tutti i file dei ragni nascosti in una volta sola, senza toccare i pipistrelli o altri file.

Ho concluso con `gsh check` e la missione è stata completata con successo.

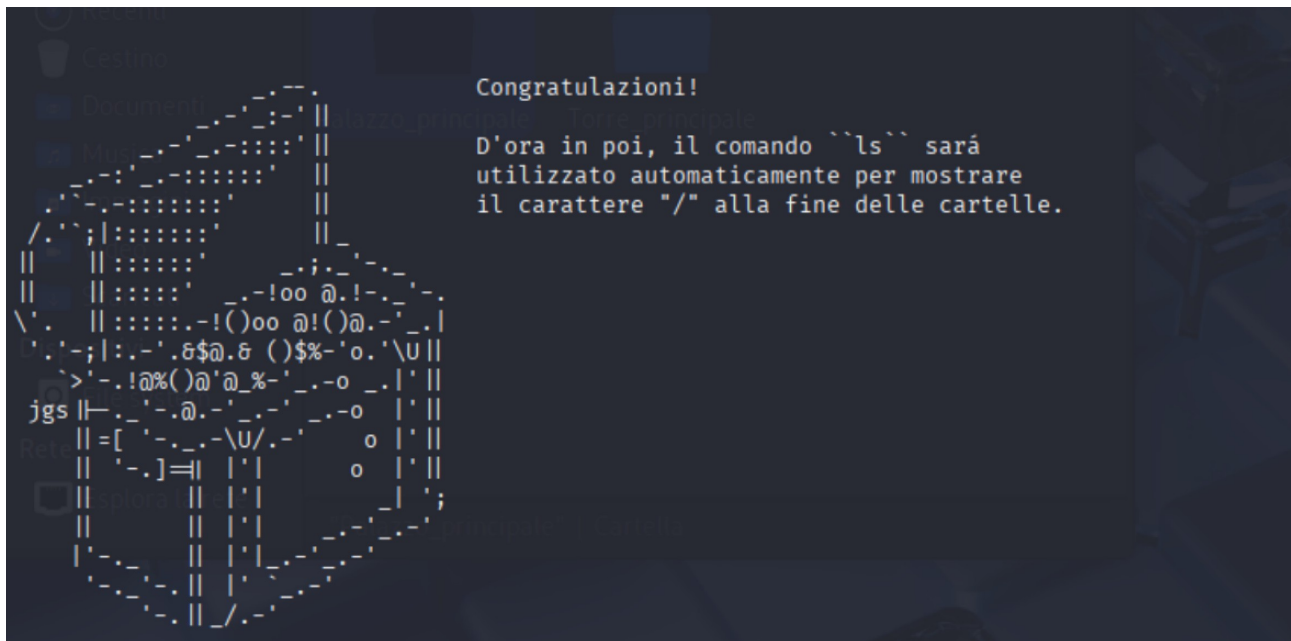
```
~/Castello/Cantina
[mission 9] $ ls -A
.102_ragno_24      .18201_ragno_14    .28435_ragno_41    .5690_ragno_23
.10433_ragno_35    .1856_ragno_27     .2891_ragno_15     .6192_pipistrello_5
.10545_ragno_25    .19727_ragno_5     .28961_ragno_8     .6323_ragno_7
10808_pipistrello_1 .19747_pipistrello_2 .29138_ragno_10    .7518_pipistrello_3
.1118_ragno_38     .2073_ragno_4      .30314_ragno_31    .7666_ragno_19
.11393_ragno_22    .22131_ragno_50    307_pipistrello_5  .7982_ragno_18
.11489_ragno_9     .22627_ragno_11    .30812_ragno_16    .8457_ragno_42
.14711_ragno_12    .23517_ragno_46    .31012_ragno_17    .9061_ragno_3
.14872_ragno_33    24585_pipistrello_3 .31304_ragno_26    .9171_ragno_48
.15612_pipistrello_1 2513_pipistrello_4 32583_pipistrello_2 .9209_pipistrello_4
.16120_ragno_44    .25254_ragno_20    .32666_ragno_6     .9307_ragno_45
.16167_ragno_30    .25779_ragno_28    .362_ragno_34      .9558_ragno_37
.16471_ragno_40    .26006_ragno_29    .3715_ragno_21     botte_di_mele
.16547_ragno_49    .26182_ragno_32    .4251_ragno_2
.17877_ragno_36    .26849_ragno_13    .4463_ragno_39
.17966_ragno_1     .2766_ragno_43     .5061_ragno_47
```

```
~/Castello/Cantina
[mission 9] $ rm .ragno*
rm: impossibile rimuovere '.ragno*': File o directory non esistente
```

```
~/Castello/Cantina
[mission 9] $ rm .*_ragno_*
```

```
~/Castello/Cantina
[mission 9] $ gsh check
```

```
Congratulazioni, la missione 9 è stata completata con successo!
```

Comandi usati in ordine

```
ls -A
rm .ragno*           # (tentativo fallito)
rm .*_ragno_*       # comando corretto
gsh check
```

Missione 10 – I quattro stendardi

L'obiettivo era recuperare i **quattro stendardi** dalla **Grande_sala** del Castello, senza rubarli. Dovevo semplicemente **copiarli** nella mia **Cassa**, che si trova dentro la **Capanna** nella **Foresta**, mantenendo **stesso nome e contenuto**.

Mi trovavo nella **Cantina**, quindi sono salito fino alla **Grande_sala**. Ho eseguito `ls` per vedere i file presenti e ho trovato i quattro stendardi: `stendardo_1`, `stendardo_2`, `stendardo_3` e `stendardo_4`.

Ho provato inizialmente a eseguire `stendardo_1 ~/Foresta/Capanna/Cassa`, ma ho ricevuto un errore (comando non trovato) perché non avevo usato il comando giusto.

Ho corretto usando il comando `cp`, che serve a **copiare file**. È l'abbreviazione di **"copy"** e permette di spostare una copia del file in un'altra cartella, lasciando intatto l'originale.

Ho copiato uno a uno tutti i file dei quattro stendardi nella mia **Cassa**.

Dopo aver controllato che tutto fosse al posto giusto, ho lanciato `gsh check` e la missione è stata completata con successo.

```
File Azioni Modifica Visualizza Aiuto
[mission 10] $ ls
28201_testa_di_cervo    54549_armatura    stendardo_2    stendardo_4
43755_scudo_decorativo  stendardo_1        stendardo_3
Posizioni
~/Castello/Grande_sala
[mission 10] $ stendardo_1 ~/Foresta/Capanna/Cassa
stendardo_1: comando non trovato
~/Castello/Grande_sala
[mission 10] $ ls
28201_testa_di_cervo    54549_armatura    stendardo_2    stendardo_4
43755_scudo_decorativo  stendardo_1        stendardo_3
~/Castello/Grande_sala
[mission 10] $ cp stendardo_1 ~/Foresta/Capanna/Cassa
~/Castello/Grande_sala
[mission 10] $ cp stendardo_2 ~/Foresta/Capanna/Cassa
~/Castello/Grande_sala
[mission 10] $ cp stendardo_3 ~/Foresta/Capanna/Cassa
~/Castello/Grande_sala
[mission 10] $ cp stendardo_4 ~/Foresta/Capanna/Cassa
~/Castello/Grande_sala
[mission 10] $ gsh check
Congratulazioni, la missione 10 è stata completata con successo!
```

Comandi usati:

```
cd ~/Castello/Grande_sala
ls
stendardo_1 ~/Foresta/Capanna/Cassa    # tentativo errato
cp stendardo_1 ~/Foresta/Capanna/Cassa
cp stendardo_2 ~/Foresta/Capanna/Cassa
cp stendardo_3 ~/Foresta/Capanna/Cassa
cp stendardo_4 ~/Foresta/Capanna/Cassa
gsh check
```

Report riepilogativo dei comandi usati fino a missione 10

Dopo un primo blocco di attività, ho deciso di fermarmi per fare il punto sui comandi che ho usato fino ad ora. Ho incontrato situazioni molto diverse: spostamenti tra cartelle, creazione di oggetti, rimozione selettiva di file, gestione di file nascosti, spostamenti rapidi con percorsi completi e altro ancora.

Molti dei comandi sono ormai familiari, altri li ho usati in modo più mirato. Alcuni errori iniziali (come cercare di eseguire un file come se fosse un comando) mi hanno aiutato a capire meglio la sintassi della shell.

Per organizzare meglio quello che ho imparato, ho messo insieme una tabella che raccoglie i **comandi usati e il loro significato**, così posso rivederli facilmente quando voglio. È un modo pratico per fissare ciò che so, e prepararmi ai comandi più avanzati che arriveranno dopo.

Comando	Scopo / Perché è stato usato
<code>cd NOME_CARTELLA</code>	Entra in una directory specifica
<code>cd ..</code>	Torna indietro di un livello nella struttura delle cartelle
<code>cd</code>	Torna direttamente alla directory iniziale (home)
<code>cd PERCORSO_COMPLETO</code>	Spostamento diretto da un punto qualsiasi a una destinazione precisa
<code>ls</code>	Mostra il contenuto della cartella attuale
<code>ls -A</code>	Mostra anche i file nascosti (quelli che iniziano con <code>.</code>)
<code>mkdir NOME_CARTELLA</code>	Crea una nuova cartella
<code>rm NOME_FILE / rm *</code>	Elimina uno o più file
<code>rm *ragno*</code>	Rimuove tutti i file che contengono "ragno" nel nome
<code>rm .*_ragno_*</code>	Rimuove tutti i file nascosti che contengono "ragno" nel nome
<code>mv FILE DESTINAZIONE</code>	Sposta un file in un'altra cartella
<code>cp FILE DESTINAZIONE</code>	Copia un file in un'altra cartella
<code>gsh check</code>	Verifica se l'obiettivo richiesto è stato raggiunto
<code>gsh goal</code>	Mostra l'obiettivo corrente
<code>standardo_1 ... (errore)</code>	Tentativo errato di usare un file come comando

Missione 11 – Gli arazzi della Grande Sala

L'obiettivo era copiare tutti gli **arazzi** presenti nella **Grande_sala** del Castello all'interno della **Cassa**, custodita nella **Capanna** nella Foresta.

I file degli arazzi erano facilmente riconoscibili grazie alla parola `arazzo` presente nel loro nome.

Dalla **Grande_sala**, ho verificato il contenuto della cartella con `ls`, confermando la presenza dei file da copiare.

Ho poi utilizzato una wildcard per selezionare tutti i file con `arazzo` nel nome e copiarli tutti in una sola volta nella destinazione corretta.

La verifica finale con `gsh check` ha confermato che la missione è stata completata con successo.

```
~/Castello/Grande_sala
[mission 11] $ cd ~/Castello/Grande_sala

~/Castello/Grande_sala
[mission 11] $ ls
26564_arazzo_09  4003_arazzo_10      48894_arazzo_02  6515_arazzo_07  standardo_4
30898_arazzo_03  40151_arazzo_06     49124_arazzo_05  standardo_1
32466_arazzo_04  44520_testa_di_cervo 56238_armatura   standardo_2
3834_arazzo_01   47772_scudo_decorativo 630_arazzo_08    standardo_3

~/Castello/Grande_sala
[mission 11] $ cp *_arazzo_* ~/Foresta/Capanna/Cassa

~/Castello/Grande_sala
[mission 11] $ gsh check

Congratulazioni, la missione 11 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

cp serve a copiare uno o più file in un'altra cartella, mantenendo nome e contenuto.

```
cd ~/Castello/Grande_sala
```

```
ls
```

```
cp *_arazzo_* ~/Foresta/Capanna/Cassa
```

```
gsh check
```

Missione 12 – Il più vecchio dei dipinti

Nel **Primo_piano** della **Torre_principale**, ho trovato tre dipinti antichi.

Per completare la missione, ho individuato quello con la **data di h goa più vecchia** utilizzando `ls -l`. Il più datato risultava essere:

dipinto_RwwLrXUd – 15 ottobre 1987

Ho poi eseguito `cat` per ammirarlo, come richiesto dall'obiettivo secondario, e l'ho copiato nella mia **Cassa** all'interno della **Capanna**.

Questo esercizio mi è **piaciuto particolarmente**: il contenuto del dipinto era una **rappresentazione ASCII** e, da **grandissimo fan di Star Wars**, ho apprezzato il riferimento e il tono nostalgico del disegno.



Comandi usati

```
cd ~/Castello/Torre_principale/Primo_piano
ls -l
cat dipinto_RwwLrXUd
cp dipinto_RwwLrXUd ~/Foresta/Capanna/Cassa
gsh check
```

Missione 13 – La congiunzione stellare di Nostradamus

Questa volta si trattava di una data precisa: **16 novembre 1918**.

L'obiettivo era scoprire che **giorno della settimana** fosse, secondo quanto previsto da Nostradamus.

Mi è piaciuta questa missione, perché sembrava una di quelle cose curiose da nerd del tempo e dello spazio.

Ho usato `cal 1918` per stampare l'intero calendario dell'anno, poi sono andato a cercare il mese di **novembre**.

Scorrendo colonna per colonna, ho trovato il **16** sotto la colonna del **sabato**.

Una volta sicuro, ho lanciato `gsh check`, ho inserito la risposta corretta, ed è uscita la conferma: missione riuscita.

```

  1  2  3  4  5  6          1  2  3          1  2  3  4  5  6  7
 7  8  9 10 11 12 13      4  5  6  7  8  9 10      8  9 10 11 12 13 14
14 15 16 17 18 19 20     11 12 13 14 15 16 17     15 16 17 18 19 20 21
21 22 23 24 25 26 27     18 19 20 21 22 23 24     22 23 24 25 26 27 28
28 29 30 31              25 26 27 28 29 30 31     29 30

~/.Cal
  1  2  3  4  5  6          1  2          1  2  3  4  5  6  7
do lu ma me gi ve sa do lu ma me gi ve sa do lu ma me gi ve sa
      1  2  3  4  5          1  2          1  2  3  4  5  6  7
 6  7  8  9 10 11 12      3  4  5  6  7  8  9      8  9 10 11 12 13 14
13 14 15 16 17 18 19     10 11 12 13 14 15 16     15 16 17 18 19 20 21
20 21 22 23 24 25 26     17 18 19 20 21 22 23     22 23 24 25 26 27 28
27 28 29 30 31          24 25 26 27 28 29 30     29 30 31

~/.Musica
~/Castello/Torre_principale/Primo_piano
[mission 13] $ gsh check
Quale é la data per il giorno della settimana per 11-16-1918?
 1 : lunedì
 2 : martedì
 3 : mercoledì
 4 : giovedì
 5 : venerdì
 6 : sabato
 7 : domenica
La tua risposta: 6

Congratulazioni, la missione 13 é stata completata con successo!
```

Comandi usati

cal 1918

6

gsh check

Missione 14 – Scorciatoia nascosta

Questa volta si trattava di semplificarsi la vita: creare un alias per non dover più scrivere `ls -A` ogni volta che voglio vedere anche i file nascosti.

Ho creato l'alias `la`, assegnandogli proprio il comando `ls -A`. L'ho testato subito e ha funzionato al primo colpo.

Tra i file elencati è apparsa **.bella_roccia**, ben mimetizzata, ma ora ben visibile.

Ho completato la missione con `gsh check`, ed è stata confermata senza problemi.

Una cosa semplice, ma super comoda.

```
~/Castello/Torre_principale/Primo_piano
[mission14] $ alias la='ls -A'

~/Castello/Torre_principale/Primo_piano
[mission 14] $ la
.bella_roccia  dipinto_AqdBztio  dipinto_fFmjbbWY  dipinto_RwwLrXUd

~/Castello/Torre_principale/Primo_piano
[mission 14] $ gsh check

Congratulazioni, la missione 14 é stata completata con successo!
```

Comandi usati

alias la='ls -A'

la

gsh check

Missione 15 - Primo appunto nel diario

In questa missione ho creato un file chiamato `registro.txt` all'interno della cartella della Cassa, con lo scopo di iniziare a usarlo come diario personale per le missioni future.

Mi sono spostato nella directory `~/Foresta/Capanna/Cassa` e ho aperto il file usando l'editor `nano`. Al suo interno ho scritto la frase:

Questo è il diario di ADR

Ho poi salvato il file con `Control + O`, confermato con `Invio` e infine chiuso con `Control + X`.

Tornato nella shell, ho eseguito il comando `gsh check` per verificare che tutto fosse andato a buon fine. La missione è risultata completata con successo.

```
~/Castello/Torre_principale/Primo_piano
[mission 15] $ cd ~/Foresta/Capanna/Cassa
~/Foresta/Capanna/Cassa
[mission 15] $ nano registro.txt
~/Foresta/Capanna/Cassa
[mission 15] $ gsh check
Congratulazioni, la missione 15 é stata completata con successo!
[ progress was saved in /home/kali/gameshell-save.sh ]
```

Comandi usati

```
cd ~/Foresta/Capanna/Cassa
nano registro.txt
gsh check
```

Missione 16 - Alias per modificare lo storico

Questa volta ho creato un alias chiamato "storico" per poter aprire e modificare facilmente il file storico.txt all'interno della Cassa, da qualsiasi posizione mi trovi nel terminale.

Ho usato il comando alias per collegare la parola "storico" al comando completo con il percorso del file.

Dopo aver lanciato il comando, ho scritto all'interno del file la frase:

Questo è il mio file storico ADR

L'ho salvato con Control + O, confermato con Invio, e chiuso con Control + X.

Infine ho eseguito gsh check, che ha confermato il completamento della missione.

```
Congratulazioni, la missione 16 é stata completata con successo!
Complimenti!
Da ora puoi utilizzare il file
"~/gshrc"
per registrare gli alias.
Altre informazioni possono essere
trovate nel file (è stato creato per te).
```

Comandi usati

```
alias storico='nano ~/Foresta/Capanna/Cassa/storico.txt'
storico
gsh check
```

Missione 17 - La regina dei ragni

Questa è stata senza dubbio la missione più snervante affrontata fino a questo punto. L'obiettivo era apparentemente semplice: entrare nella tana nascosta della regina dei ragni e cancellare il file che la rappresentava, tutto entro 20 secondi. Nessuna wildcard disponibile, quindi vietato l'uso di * o ?, solo precisione e velocità. E ovviamente, non bisognava toccare i pipistrelli.

La sfida non era solo nella velocità, ma soprattutto nella gestione dei **nomi dei file e delle cartelle**, generati con stringhe lunghissime e piene di caratteri strani, in particolare **spazi non standard** (probabilmente Unicode), che rendevano impossibile scrivere correttamente il percorso a mano. A quel punto ho capito che l'unica soluzione era **affidarsi completamente al completamento automatico con il tasto Tab**.

Dopo aver fatto numerosi reset (`gsh reset`), ogni tentativo diventava un allenamento a essere più rapido.

Ho dovuto tentare **sette volte** prima di riuscire a completare tutto in meno di 20 secondi. Ecco cosa succedeva:

- A volte la regina veniva eliminata correttamente ma il tempo era troppo lungo (27, 50, anche 115 secondi).
- Altre volte sbagliavo percorso o mi confondevo con la digitazione a causa dei nomi lunghissimi.
- In un'occasione ho anche digitato `re` invece di `rm`, avviando un'installazione totalmente inutile.
- L'unico modo per riuscire davvero era **imparare a riconoscere velocemente il nome giusto**, entrare nella cartella con `cd + Tab`, identificare il file con `ls`, e rimuoverlo in un colpo solo.

Alla fine, con un ultimo sprint, ho cancellato la regina in **16 secondi netti**.

Quando ho letto "Perfetto, hai impiegato solo 16 secondi", ho quasi applaudito da solo davanti allo schermo.

Questa missione mi ha fatto rendere conto di quanto i comandi più semplici (come `cd`, `ls`, `rm`, `Tab`) diventino essenziali **quando ogni secondo conta**, e soprattutto quando il sistema stesso gioca con i caratteri per metterti in difficoltà.

```
~/Castello/Cantina
[mission 17] $ cd .Tana_della_regina_dei_ragni\ nXYUVPzKctlQNSMU MdufeZkzpkqKUToy/

~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 17] $ ls
PeiRfqtEIrZZNUQ_regina_dei_ragni_pINYiAIhdGVtTrYk
woWJMSETOLJSgwBh_piccolo_pipistrello_BWFlFyEiPSGONKax

~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 17] $ rm PeiRfqtEIrZZNUQ_regina_dei_ragni_pINYiAIhdGVtTrYk

~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 17] $ gsh check
Perfetto, hai impiegato solo 16 secondi per completare questa missione!

Congratulazioni, la missione 17 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```


Comandi usati

```
gsh reset
cd ~/Castello/Cantina
ls -A
cd .Tana_della_regina_dei_ragni<Tab>
ls
rm nome_regina_dei_ragni<Tab>
gsh check
```

Giudizio finale:

Missione davvero tosta, ma anche tra le più soddisfacenti una volta completata. È servita pazienza, riflessi e controllo del terminale. Esperienza memorabile peggiore di quella volta in cui ho dovuto uccidere **Nemesis** a **Resident Evil** solo con la pistola!!!

Missione 18 – Gli occhi ti osservano

Dopo l'intensità della missione precedente, questa è stata una vera boccata d'aria. Mi è stato chiesto di eseguire il comando `xeyes`, un'applicazione grafica che mostra due occhi animati che seguono ogni movimento del cursore del mouse. A dirla tutta, è stato divertente e anche un po' inquietante all'inizio.

Ho avviato il comando normalmente:

```
xeyes
```

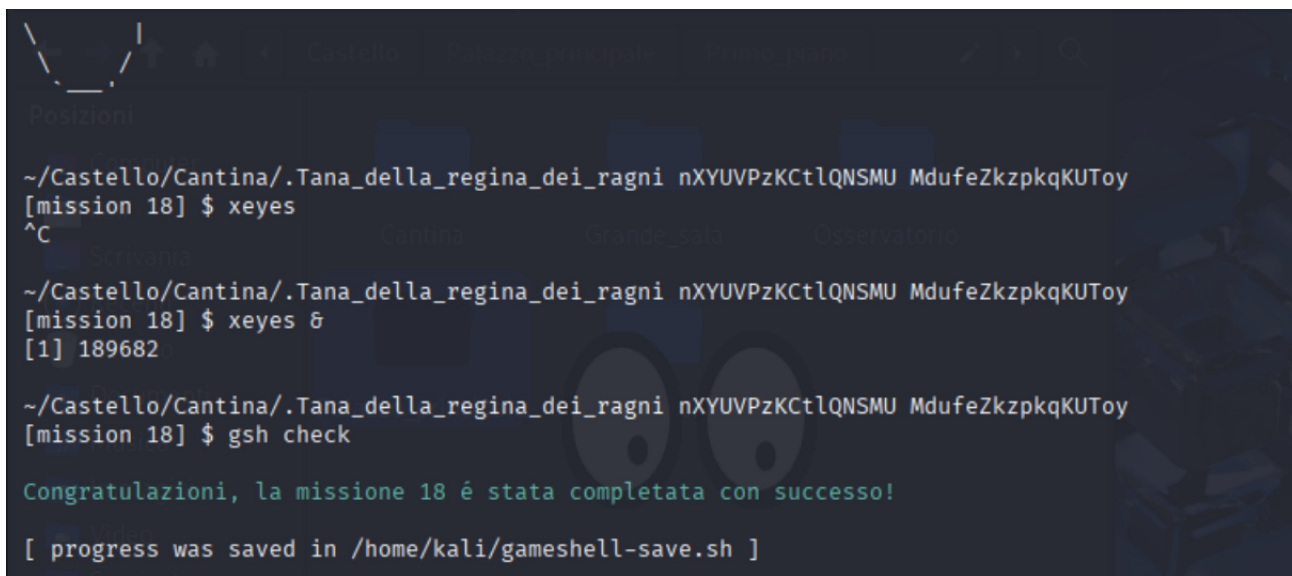
Dopo qualche secondo a fissarmi, li ho fermati con `Ctrl + C`.

Poi li ho rilanciati in background:

```
xeyes &
```

E da quel momento... non mi hanno più lasciato. Quegli occhietti mi stanno **ancora guardando mentre continuo a lavorare nel terminale**, come se stessero seguendo ogni mio passo.

Li trovo **simpaticissimi**, al punto che mi viene voglia di tenerli lì anche mentre continuo il mio viaggio in GameShell.



```
~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 18] $ xeyes
^C
~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 18] $ xeyes &
[1] 189682

~/Castello/Cantina/.Tana_della_regina_dei_ragni nXYUVPzKctlQNSMU MdufeZkzpkqKUToy
[mission 18] $ gsh check

Congratulazioni, la missione 18 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Comandi usati

xeyes

Ctrl + C

xeyes &

gsh check



Missione 19: Fuochi d'artificio per il pirotecnico

All'inizio sembrava una delle missioni più semplici: “accendere almeno 3 fuochi d'artificio così che il pirotecnico del re potesse vederli”. Facile a dirsi, meno a farsi.

Ho provato diverse volte, cambiando posizione e combinazione di comandi. Prima li ho lanciati in sequenza con:

```
flarigo ; flarigo ; flarigo
```

Poi ho provato in background:

```
flarigo & flarigo & flarigo
```

Ma ogni volta, niente da fare. Il pirotecnico non vedeva nulla. Probabilmente i fuochi partivano troppo presto o non tutti assieme.

Ho anche tentato con un ritardo di 5 secondi tra l'uno e l'altro:

```
sleep 5; flarigo; sleep 5; flarigo; sleep 5; flarigo
```

E anche con esecuzione parallela e ritardo:

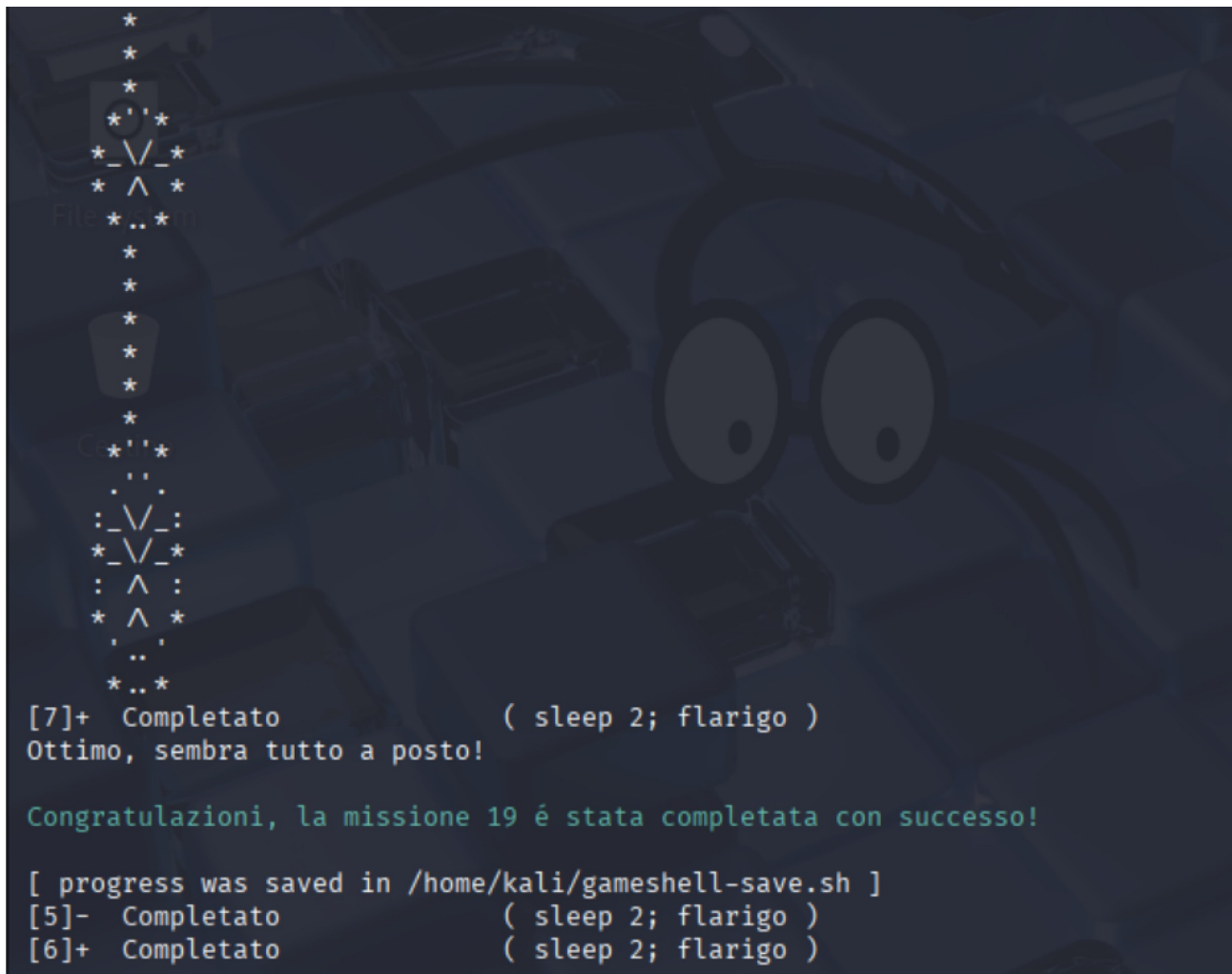
```
(sleep 5; flarigo) & (sleep 5; flarigo) & (sleep 5; flarigo) &
```

Nulla. Ancora “non vede niente”.

Alla fine ho capito che bisognava dare **un piccolo ritardo uniforme prima che partissero tutti in contemporanea**, e solo allora ho lanciato il comando vincente:

```
gsh g
(sleep 2; flarigo) & (sleep 2; flarigo) & (sleep 2; flarigo) &
```

E finalmente... BOOM! Il cielo si è acceso, il pirotecnico ha visto tutto, e **la missione è stata completata con successo!**



Comandi utilizzati:

```
flarigo ; flarigo ; flarigo
flarigo & flarigo & flarigo
sleep 5; flarigo; sleep 5; flarigo; sleep 5; flarigo
(sleep 5; flarigo) & (sleep 5; flarigo) & (sleep 5; flarigo) &
(sleep 2; flarigo) & (sleep 2; flarigo) & (sleep 2; flarigo) &
gsh check
```

20 Missione: Incantesimo per il gran finale

Questa era una di quelle missioni che sembrano semplici ma ti fregano subito. Dovevo aiutare il pirotecnico del re a ricordare l'incantesimo giusto, che iniziava con charmiglio seguito da quattro lettere. Sì, ok, facile... ma quali lettere?

Ho provato subito:

```
charmiglio abcd
```

Risultato? Un'esplosione di simboli senza senso e una magia che non partiva mai. Interrotto con ^C come suggerito.

Poi ci ho riprovato:

charmiglio qwer

Stessa storia. Una tempesta di caratteri, effetti grafici... e niente.

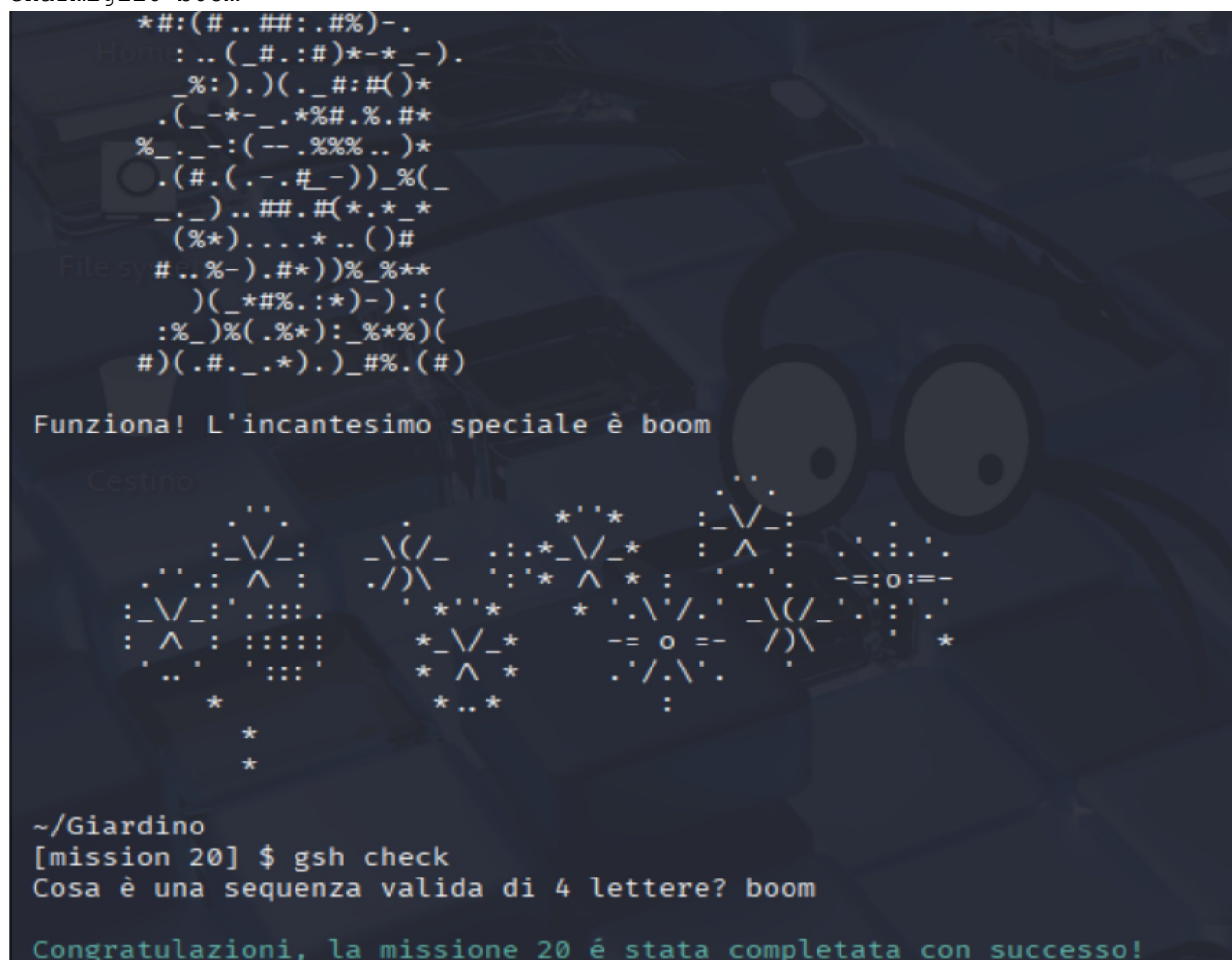
Non mollando, ci ho buttato dentro anche:

charmiglio fire

Pensavo fosse la svolta, e invece... solo altri effetti scenici inutili.

Alla fine ho provato qualcosa di più “tematico”:

charmiglio boom



Ed ecco che partono fuochi veri, esplosioni e luci. Il pirotecnico contentissimo, io pure. Finalmente!

Comandi usati:

```
charmiglio abcd  
charmiglio qwer  
charmiglio fire  
charmiglio boom  
gsh check
```

BREAK2

Dalla missione 11 alla 20 le cose hanno iniziato a farsi più interessanti e un po' più complesse. All'inizio tutto filava liscio: copiare file, creare alias, scrivere nei file... insomma, gestibile. Poi però sono arrivati i momenti tosti: la missione della regina dei ragni è stata una vera corsa contro il tempo — entrare in quella cartella con un nome sempre diverso era snervante! I fuochi d'artificio (missione 19) mi hanno fatto impazzire: lanciavo e rilanciavo, ma niente, finché non ho trovato il trucco del ritardo perfetto. E l'incantesimo "boom"? Ci ho provato di tutto prima di beccare la combinazione giusta. Ma alla fine, con pazienza (e un po' di testardaggine), ce l'ho fatta!

Tabbella comandi missioni 11-20

Comando	Spiegazione a parole mie
<pre>cp *_arazzo_* ~/Foresta/Capanna/Cassa</pre>	Dovevo copiare gli arazzi nella cassa. Uso la wildcard per beccare tutti quelli col nome giusto.
<pre>ls -l</pre>	Mi serviva per vedere le date dei dipinti e capire qual era il più vecchio.
<pre>cat dipinto_RwwLrXUd</pre>	Curiosità personale: volevo vedere cosa c'era dentro il file, era troppo nerd per ignorarlo.
<pre>cp dipinto_RwwLrXUd ~/Foresta/Capanna/Cassa</pre>	Copiato il dipinto più vecchio nella mia cassa, come da richiesta.
<pre>cal 1918</pre>	Dovevo scoprire che giorno era il 16 novembre 1918. Con questo comando ho visto il calendario.
<pre>gsh check</pre>	Sempre il comando per verificare se ho completato la missione (ripetuto ogni volta).
<pre>alias la='ls -A'</pre>	Missione carina: mi chiedeva di creare un alias per visualizzare i file nascosti in due lettere.
<pre>la</pre>	Usato subito dopo per testare l'alias, funzionava alla grande.
<pre>cd ~/Foresta/Capanna/Cassa</pre>	Mi serviva per entrare nella cartella dove salvare o creare file.
<pre>nano registro.txt</pre>	Ho creato il mio diario personale della missione. Sì, ho scritto proprio "Questo è il diario di ADN".
<pre>alias storico='nano ~/Foresta/Capanna/Cassa/storico.txt'</pre>	Per aprire il file storico da ovunque senza riscrivere il path intero ogni volta.

Comando	Spiegazione a parole mie
<code>ls -A</code>	Fondamentale per vedere la directory nascosta con nome lunghissimo in missione 17.
<code>cd .Tana_della_regina_dei_ragni\ ...</code>	Ogni volta era diverso il nome e ogni volta un'impresa capire come entrarci!
<code>rm <nome_file_regina></code>	Dovevo rimuovere solo la regina dei ragni, quindi il comando <code>rm</code> su un file preciso.
<code>gsh reset</code>	Per azzerare il tempo e riprovare più veloce. Missione con cronometro stressante.
<code>xeyes</code>	Lanciato per la missione degli occhi. Comando simpatico, davvero.
<code>^C (Control-C)</code>	Usato più volte per interrompere <code>xeyes</code> o incantesimi falliti.
<code>xeyes &</code>	L'ho mandato in background. Gli occhi mi guardavano anche mentre giocavo.
<code>flarigo ; flarigo ; flarigo</code>	Primo tentativo: uno dopo l'altro. Non bastava.
<code>flarigo & flarigo & flarigo</code>	Poi li ho mandati in background, ma partivano troppo sfasati.
<code>(sleep 5; flarigo) & (sleep 5; flarigo) & (sleep 5; flarigo) & (sleep 5; flarigo) &</code> <code>(sleep 2; flarigo) & (sleep 2; flarigo) & (sleep 2; flarigo) & (sleep 2; flarigo) &</code>	Tentativo con ritardo, pensavo fosse l'idea giusta. Macché.
<code>charmiglio abcd / qwer / fire / boom</code>	Questo è stato il comando vincente. Piccolo ritardo, ma esplosione simultanea.
	Ho provato varie combinazioni per l'incantesimo. Solo "boom" ha funzionato. ?

Missione 21: La moneta nel labirinto

Questa sembrava una di quelle missioni facili facili... "Trova la moneta e portala nella tua cassa". Ma no, ovviamente no. Il labirinto era un groviglio assurdo di cartelle con nomi incomprensibili, tipo codici seriali di qualche piano top secret.

Mi sono messo a scavare con `ls -R`, ma dopo un po' mi sono stufato e ho cercato di andare più diretto usando un comando per trovare direttamente dove fosse 'sta benedetta moneta. Alla fine l'ho vista, sepolta dentro mille sottocartelle.

A quel punto, ho voluto fare il figo e spostarla usando una variabile, ma lì ho sbagliato perché avevo provato a usare `alias` al posto della variabile. Ovviamente il comando non funzionava e ci ho perso un po' a capire dov'era il problema.

Quando finalmente ho sistemato tutto, ho spostato la moneta dentro la mia cassa e voilà: missione completata. Soddisfazione totale. Sembrava facile, ma mi ha fregato con quei nomi assurdi.

```
File Azioni Modifica Visualizza Aiuto
~/Giardino/Labirinto
[mission 21] $ mv $moneta ~/Foresta/Capanna/Cassa

~/Giardino/Labirinto
[mission 21] $ gsh check

Congratulazioni, la missione 21 é stata completata con successo!

File system

Congratulazioni!

Da ora, il comando ``ls`` proverà ad usare un codice di colori per identificare più facilmente fra cartelle e diversi tipi di file.
```

Comandi utilizzati:

- `cd ~/Giardino/Labirinto`
- `ls -R`
- `ls -RA`
- `find ~/Giardino/Labirinto -name "*moneta*"`
- `alias moneta='~/Giardino/.../00000_moneta_di_rame_00000'` (che non funzionava)
- `moneta=~/Giardino/.../00000_moneta_di_rame_00000` (quello corretto)
- `mv $moneta ~/Foresta/Capanna/Cassa`
- `gsh check`

Missione 22: Moneta d'argento nel labirinto

Dopo il rame, tocca all'**argento**. E già sapevo che sarebbe stato un altro bel girotondo tra directory con nomi assurdi.

Come sempre, sono partito con un bel `find` per stanare la benedetta moneta. E lì già si capiva: il nome era `00000_moneta_dargento_00000` (senza "di", ovviamente... perché mai usare l'italiano corretto?). All'inizio avevo messo il nome sbagliato e pure il percorso era finto. Insomma: errore su errore, e `mv` che mi rispondeva con il solito "file non trovato". Classico.

Una volta corretto tutto, ho salvato il percorso vero in una variabile `moneta`, e da lì... è filato tutto liscio! Moneta spostata nella mia cassa e missione completata con tanto di congratulazioni animate.

```

File Azioni Modifica Visualizza Aiuto
[mission 22] $ moneta=~/.Giardino/Labirinto/percorso/completo/00000_moneta_di_argento_00000

~/.Giardino/Labirinto
[mission 22] $ mv $moneta ~/.Foresta/Capanna/Cassa
mv: impossibile eseguire stat di '/home/kali/gameshell.1/World/Giardino/Labirinto/percorso/completo/00000_moneta_di_argento_00000': File o directory non esistente

~/.Giardino/Labirinto
[mission 22] $ find ~/.Giardino/Labirinto -name "*argento*"
/home/kali/gameshell.1/World/Giardino/Labirinto/04f24892a64ad8b/c3a4b45403f291c74b845ab/dd124320fcc60cd8bb2180975b1b/00000_moneta_dargento_00000

~/.Giardino/Labirinto
[mission 22] $ /home/kali/gameshell.1/World/Giardino/Labirinto/xxxxx/yyyy/zzzz/00000_moneta_di_argento_00000
bash: /home/kali/gameshell.1/World/Giardino/Labirinto/xxxxx/yyyy/zzzz/00000_moneta_di_argento_00000: File o directory non esistente

~/.Giardino/Labirinto
[mission 22] $ moneta=/home/kali/gameshell.1/World/Giardino/Labirinto/04f24892a64ad8b/c3a4b45403f291c74b845ab/dd124320fcc60cd8bb2180975b1b/00000_moneta_dargento_00000

~/.Giardino/Labirinto
[mission 22] $ mv $moneta ~/.Foresta/Capanna/Cassa

~/.Giardino/Labirinto
[mission 22] $ gsh check

Congratulazioni, la missione 22 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]

```

Comandi Utilizzati

```
find ~/.Giardino/Labirinto -name "*argento*"
```

```
moneta=/home/kali/gameshell.1/World/Giardino/Labirinto/04f24892a64ad8b/c3a4b45403f291c74b845ab/dd124320fcc60cd8bb2180975b1b/00000_moneta_dargento_00000
```

```
mv $moneta ~/.Foresta/Capanna/Cassa
```

```
gsh check
```

Missione 23 – Monete d'oro nel labirinto

Questa volta mi è toccata una missione “a gruzzoli”: trovare **tutte** le monete d'oro nascoste nel labirinto e portarle nella mia cara cassaforte personale.

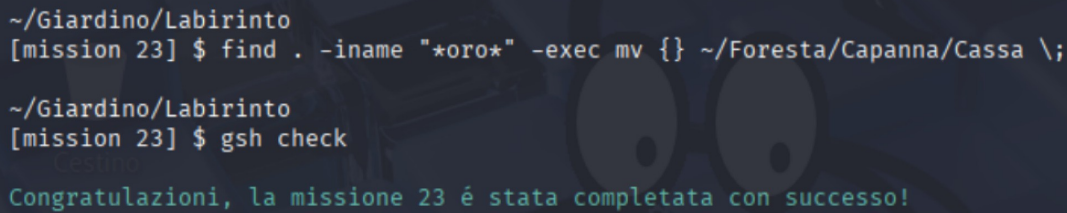
All'inizio sembrava che ci fosse solo una, ma poi ho capito subito che erano **più di una**. Ho iniziato con il classico:

```
find . -iname "*oro*"
```

Ed ecco comparire due bei risultati, con nomi diversi e in posizioni un po' infognate. Quindi ho cercato il metodo più veloce per **trasportarle entrambe** con un unico colpo. Dopo un attimo di tentennamento e un paio di comandi scritti male (eh sì, mi scappa ancora qualche simbolo inutile...), ho riscritto bene il comando magico:


```
find . -iname "*oro*" -exec mv {} ~/Foresta/Capanna/Cassa \;
```

Una riga sola e via, tutte le monete trasportate in cassa in un battito di ciglia. E boom – **missione completata con successo**.



```
~/Giardino/Labirinto  
[mission 23] $ find . -iname "*oro*" -exec mv {} ~/Foresta/Capanna/Cassa \;  
  
~/Giardino/Labirinto  
[mission 23] $ gsh check  
  
Congratulazioni, la missione 23 è stata completata con successo!
```

Comandi utilizzati:

```
find . -iname "*oro*"
find . -iname "*oro*" -exec mv {} ~/Foresta/Capanna/Cassa \;
gsh check
```

Missione 24 – Il tè di Servillus nella grotta

Questa volta niente monete strane o labirinti assurdi. Il compito sembrava semplice: trovare una ricetta per un tè alle erbe per un anziano smemorato, Servillus, nascosto nella sua grotta. Ovviamente, conoscendo già GameShell, ero certo che dietro questa apparente semplicità ci fosse qualche piccolo tranello.

Parto subito a navigare tra le varie directory per capire dove sia esattamente questa famosa grotta. Controllo con `ls` e trovo diverse cartelle tipo Castello, Foresta, Giardino, Montagna e Stalla. La grotta sembrava stare sulla Montagna, quindi mi sposto lì con il classico:

```
cd Montagna/Grotta
```

Arrivo dentro e trovo la cartella chiamata `Libro_delle_pozioni`. Ovviamente, dentro c'è un bel casino: 13 pagine sparpagiate con nomi tipo `pagina_01`, `pagina_02`, e via dicendo fino alla `pagina_13`.

Do un'occhiata veloce al contenuto delle pagine, usando il comando più rapido possibile:

```
cat pagina_07
```

Perfetto! Alla pagina 7 salta fuori la ricetta che mi serviva ("Tè d'erbe"). C'è solo un piccolo problema: dopo la ricetta appare anche un testo inutile riguardo un fantomatico mantello dell'invisibilità.

Ora ricordo bene la missione: per superarla devo visualizzare SOLO titolo e ricetta, niente altro. Per fortuna mi torna utile il buon vecchio comando `head` che limita l'output alle prime righe. Guardo attentamente quante righe mi servono, ne conto 6, e così lancio il comando giusto:

```
head -n 6 Libro_delle_pozioni/pagina_07
```

Sembra semplice detta così, ma come sempre con GameShell, se sbagli anche solo un numero di righe, la missione non passa.

```

/ | cat FILE
/ | Visualizza il contenuto del file.
/ | head FILE
/ | Visualizza le prime 10 righe del file.
/ | head -n K FILE
/ | Stampa le prime K righe del file.
/ | Nota
/ | ____
/ |
/ | Il "FILE" potrebbe contenere cartelle se il file in questione non
/ | è nella cartella corrente.
/ |
(____)-----

Gestione
~/Montagna/Grotta
[mission 24] $ head -n 6 Libro_delle_pozioni/pagina_07
vvvvvvvvv
Tè d'erbe
^^^^^^^^^

1) Bolli dell'acqua.
2) Aggiungi erbe della foresta.
3) Lascia riposare per cinque minuti e bevi ancora caldo.

~/Montagna/Grotta
[mission 24] $ gsh check

Congratulazioni, la missione 24 è stata completata con successo!

```

Comandi utilizzati:

```

ls
cd Montagna/Grotta/Libro_delle_pozioni
cat pagina_07 (per controllare il contenuto)
head -n 6 Libro_delle_pozioni/pagina_07 (per l'output corretto)
gsh check (verifica finale missione)

```

Missione 25 – Stufato di funghi con Servillus

Durante la missione ho aiutato Servillus a preparare lo stufato di funghi rimanendo sempre nella grotta. Per prima cosa mi sono spostato nella cartella del libro delle pozioni con `cd Libro_delle_pozioni` e ho letto l'indice con `cat indice` per scoprire che lo stufato si trovava a pagina 12. Ho poi controllato il contenuto completo della ricetta tramite `cat pagina_12`. Tornato nella grotta usando `cd ..`, ho estratto soltanto i nove passi finali senza titolo e decorazioni grazie a `tail -n 9 Libro_delle_pozioni/pagina_12` e infine ho confermato il successo della missione con `gsh check`.

Comandi utilizzati: `cd Libro_delle_pozioni`, `cat indice`, `cat pagina_12`, `cd ..`, `tail -n 9 Libro_delle_pozioni/pagina_12`, `gsh check`.

Comandi utilizzati: `cat Libro_delle_pozioni/indice`,
`cat Libro_delle_pozioni/pagina_01 Libro_delle_pozioni/pagina_02`,
`gsh check`.

Missione 27 – Elisir della Giovinezza

Stavolta Servillus, vedendo la mia abilità con le pozioni, mi ha sfidato a trovare la ricetta del famoso Elisir della Giovinezza. Come sempre, prima di tutto ho consultato l'indice (`cat Libro_delle_pozioni/indice`) e ho visto che la ricetta era divisa fra le pagine 3 e 4.

Ho provato inizialmente a visualizzare le due pagine direttamente con un `cat`, ma uscivano anche righe inutili come decorazioni e avvisi vari. Così ho capito che la soluzione corretta era concatenare le due pagine e visualizzare solo gli ultimi 16 passi numerati della ricetta, usando il comando:

```
cat Libro_delle_pozioni/pagina_03 Libro_delle_pozioni/pagina_04 | tail -n 16
```

e subito dopo ho dato il comando di verifica (`gsh check`).

```
|
|--+-----+
| Usa il comando                               |
| $ gsh help                                   |
| per ottenere una lista dei comandi "gsh".    |
|--+-----+
|
File system

~/Montagna/Grotta
[mission 27] $ cat Libro_delle_pozioni/pagina_03 Libro_delle_pozioni/pagina_04 | tail -n 16
gsh check
1) Riempi un calderone con l'acqua del bagno usata.
2) Metti una rana moderatamente grande nell'acqua.
3) Lascia riposare il preparato durante la notte.
4) La mattina successiva ringrazia e libera il tuo piccolo amico verde.
5) Bolli l'acqua e aggiungi alcuni pezzi di legno di quercia.
6) Rompi 5 pietre di fiume in una polvere fine.
7) Mescola un terzo della polvere e mescola vigorosamente.
8) Lascia riposare il preparato per un giorno.
9) Aggiungi peli di coda di scoiattolo (dati volontariamente).
10) Aggiungi la rimanente polvere di pietra.
11) Mescola la pozione molto vigorosamente, in tutte le direzioni.
12) Prenditi del tempo per riposare dopo lo sforzo.
13) Riposa un po' di più.
14) Fai un pisolino, se vuoi.
15) Aggiungi alcuni aghi di larice per condire.
16) Bevi la pozione dal calderone.

Congratulazioni, la missione 27 è stata completata con successo!
```

Comandi utilizzati:

```
cat Libro_delle_pozioni/indice
```

```
cat Libro_delle_pozioni/pagina_03 Libro_delle_pozioni/pagina_04 | tail -n 16
```


Missione 28 – Acqua distillata

Questa volta Servillus era assetato e mi ha chiesto di trovare la ricetta dell'acqua distillata. Come prima cosa ho controllato nell'indice per vedere dove fosse la ricetta, e ho scoperto che si trovava alla pagina 13. Quando ho aperto la pagina però, ho visto che c'erano due ricette dentro: quella giusta e subito dopo quella della birra del re.

All'inizio ho provato con `head -n 5 | tail -n 3`, pensando bastasse per estrarre solo i passi dell'acqua distillata, ma mancava un pezzo. La terza riga della ricetta restava fuori, probabilmente per via di una riga vuota che non avevo considerato. Dopo un attimo di confusione, ho corretto portando `head` a 6 righe, così da includere anche l'ultimo passo. Con `tail -n 3` ho tenuto solo le righe numerate, evitando titolo e decorazioni. Alla fine il comando era perfetto.

```
[mission 28] $ cat Libro_delle_pozioni/pagina_13
vvvvvvvvvvvvvvvv
Acqua distillata
^^^^^^^^^^^^^^^^
1) Bolli acqua in una pentola.
2) Condensa il vapore in un contenitore fresco.
3) Aggiungi minerali per un miglior sapore (facoltativo).
vvvvvvvvvvvvvvvv
Birra del re
^^^^^^^^^^^^^^^^
1) Vai nella cantina del castello e prendine una bottiglia.
2) Servi freddo, una pinta.

~/Montagna/Grotta
[mission 28] $ head -n 5 Libro_delle_pozioni/pagina_13 | tail -n 3
^^^^^^^^^^^^^^^^
1) Bolli acqua in una pentola.
2) Condensa il vapore in un contenitore fresco.

~/Montagna/Grotta
[mission 28] $ head -n 6 Libro_delle_pozioni/pagina_13 | tail -n 3
1) Bolli acqua in una pentola.
2) Condensa il vapore in un contenitore fresco.
3) Aggiungi minerali per un miglior sapore (facoltativo).

~/Montagna/Grotta
[mission 28] $ gsh check

Congratulazioni, la missione 28 é stata completata con successo!
```

Comandi utilizzati:

```
cat Libro_delle_pozioni/indice
```

```
cat Libro_delle_pozioni/pagina_13 head -n 6 Libro_delle_pozioni/pagina_13 | tail -n 3
```

Missione 29 – Sortilegio di carbone

Appena ho letto che un demone malizioso aveva lanciato un sortilegio nel castello, ho pensato che fosse solo una trovata narrativa. Ma poi il terminale ha iniziato a impazzire: simboli ovunque, lo schermo sembrava posseduto. Per un attimo ho davvero creduto che GameShell si fosse rotto, ero quasi pronto a resettare tutto.

Questa missione mi ha spiazzato anche per un altro motivo: è stato complicatissimo selezionare e leggere bene tutto quello che appariva. Il terminale era invaso da simboli che si muovevano in continuazione, e mentre cercavo di concentrarmi sull'obiettivo, avevo la sensazione di dover battere il tempo. Un vero caos. Sembrava una lotta contro il terminale stesso, come se stesse cercando di distrarmi apposta.

Poi ho letto bene l'obiettivo: il sortilegio era un processo. Ho dato un rapido `ps` e tra i vari processi innocui è comparso lui, in bella vista: "Sortilegio". A quel punto è bastato un `kill` con il PID giusto e la schermata ha smesso di lampeggiare come se fosse in un rito demoniaco. Finalmente tutto è tornato normale.

```
~/Montagna/Grotta
[mission 29] $ ps

  PID TTY          TIME CMD
 2354 pts/0    00:00:00 zsh
 2396 pts/0    00:00:00 bash
 2455 pts/0    00:00:00 bash
601438 pts/0    00:00:00 Sortilegio
860628 pts/0    00:00:00 ps

  *#@*
  &_**/~
  !$-#

Castro

  *#@*
  &_**/~
  !$-#

~/Montagna/Grotta
[mission 29] $ kill 601438

~/Montagna/Grotta
[mission 29] $ gsh check

Congratulazioni, la missione 29 è stata completata con successo!
```

Comandi utilizzati: `ps kill 601438`

`ps | grep Sortilegio | awk '{print $1}' | xargs -r kill -9`

`gsh goal`

Missione 30 – Il sortilegio resiliente

Questa missione è stata senza dubbio la più difficile affrontata finora. Appena iniziata, mi sono trovato con il terminale infestato da processi chiamati "Sortilegio" che si moltiplicavano come se fossero un virus. Ogni volta che ne terminavo uno, ne comparivano altri due. Lo schermo era invaso da animazioni inquietanti e simboli magici che continuavano a lampeggiare, una vera e propria infestazione digitale. Sembrava una battaglia in tempo reale contro un malware.

Il primo passo è stato capire di cosa si trattasse. Ho letto bene l'intestazione e ho visto che il "sortilegio" era un processo da terminare, ma con una particolarità: il sistema voleva che prima provassi con un segnale standard, e solo dopo, se quello non funzionava, potevo usare un segnale più aggressivo. Non sapevo nulla di questi segnali, quindi ho dovuto cercare online come funzionassero i comandi `kill` con le opzioni `-s TERM` e `-9`. Ho scoperto che:

- `TERM` è il segnale di terminazione normale (gentile), che può essere ignorato dal processo.
- `KILL (-9)` è il segnale di forza bruta, che non può essere ignorato e termina il processo all'istante.

Intanto i Sortilegi aumentavano, e io mi rendevo conto che scrivere i comandi manualmente uno alla volta sarebbe stato inutile. Così ho cercato un comando unico che potesse individuarli tutti e ucciderli. Ma per evitare errori, ho dovuto scrivermi tutta la procedura su un foglio Word, preparandola riga per riga, prima di eseguirla nel terminale. Una vera e propria lotta contro il tempo e contro la propagazione del virus.

Il comando finale che ho usato è stato:

```
ps | grep Sortilegio | awk '{print $1}' | xargs -r kill -9
```

Spiegazione dettagliata:

- `ps` elenca tutti i processi attivi nella shell.
- `grep Sortilegio` filtra solo quelli chiamati "Sortilegio".
- `awk '{print $1}'` estrae solo il primo campo (il PID).
- `xargs -r kill -9` prende tutti quei PID e li passa a `kill -9`, che li termina all'istante.

Ma la missione non sarebbe stata superata se prima non avessi provato il comando con `TERM`:

```
ps | grep Sortilegio | awk '{print $1}' | xargs -r kill -s TERM
```

Che serve a tentare una terminazione "civile" dei processi.

Durante la missione, ho fatto diversi tentativi sbagliati: errori di battitura come `Sortips` invece di `Sortilegio`, comandi `kill` dati con argomenti sbagliati, e segnali rifiutati da `GameShell`. Ogni tentativo fallito faceva crescere l'ansia, perché i processi si moltiplicavano e sembrava che il terminale stesse per esplodere.

Solo dopo aver seguito alla lettera la procedura richiesta — prima `TERM`, poi `KILL` — sono riuscito a eliminare tutti i sortilegi. Quando ho dato `gsh check` e ho visto il messaggio di successo, è stata una liberazione.

```

ps | grep Sortilegio | awk '{print $1}' | xargs -r kill -s TERM
*#@*
*_**/~
!$-#

*#@*
*_**/~
!$-#

File system

ps | grep Sortile

gio | awk '{print $1}' | xargs -r kill -9 xargs -r kill -9
kill: (4929): Argomento non valido
kill: (5427): Argomento non valido

~
[mission 30] $

*#@*
*_**/~
!$-#

ps | grep Sortileps | grep Sortilegio | awk '{print $1}' | xargs -r kill -9

~
[mission 30] $ gsh check

Congratulazioni, la missione 30 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]

```

Comandi utilizzati: ps

ps | grep Sortilegio

awk '{print \$1}'

xargs -r kill -s TERM

xargs -r kill -9

Break post-Missione 30

Onestamente, non avrei mai pensato di arrivare al punto di dover **studiare i sortilegi di un demone** come parte di un'esercitazione tecnica. Invece eccoci qui, dopo ore passate a fronteggiare processi che si moltiplicano come virus, a districarci tra PID, segnali TERM e KILL, e a cercare di non eliminare la fata per sbaglio.

Sono **più di 12 ore** che sto lavorando su **GameShell**, e la stanchezza inizia a farsi sentire. Forse sarà il periodo — siamo vicini a **Pasqua**, l'aria è mezza sospesa tra festività e studio — ma ogni comando sembra un incantesimo, ogni errore un colpo di bacchetta sbagliato.

Il fatto è che non è solo un gioco o un esercizio: questa cosa fa parte del nostro **esame su GameShell**, e ogni comando che lanciamo è una prova vera. E mentre combatto con processi che si rigenerano da soli e terminali che sembrano infestati, inizio davvero a chiedermi se tutto questo non sia anche una metafora ben riuscita di come funziona il debugging nella vita reale.

PS per un NERD come me avere un esame come questo vuol dire arrivare fino alla fine come uno dei peggiori **Boss dei Souls**.

Comandi utilizzati missioni 21-30

alias moneta='~/Giardino/.../00000_moneta_di_rame_00000'

Crea un alias temporaneo che permette di eseguire un comando o accedere a un file con un nome più breve.

awk '{print \$1}'

Estrae il primo campo (di solito il PID) da ogni riga, utile dopo ps o grep per isolare i numeri dei processi.

cat Libro_delle_pozioni/indice

Visualizza tutto il contenuto del file `indice`, senza formattazione.

cat Libro_delle_pozioni/pagina_01 Libro_delle_pozioni/pagina_02

Concatena due file e mostra tutto il contenuto in un colpo solo.

cat Libro_delle_pozioni/pagina_03 Libro_delle_pozioni/pagina_04 | tail -n 16

Concatena due file e visualizza solo le ultime 16 righe del contenuto, utile per ignorare intestazioni o decorazioni.

cat Libro_delle_pozioni/pagina_07

Visualizza il contenuto completo della pagina 07.

cat Libro_delle_pozioni/pagina_12

Mostra tutto il contenuto della pagina 12.

cat Libro_delle_pozioni/pagina_13

Mostra tutto il contenuto della pagina 13 (contiene due ricette).

cd ~/Giardino/Labirinto

Cambia directory spostandosi nella cartella indicata con percorso assoluto.

cd Libro_delle_pozioni

Accede alla directory `Libro_delle_pozioni`.

cd Montagna/Grotta/Libro_delle_pozioni

Naviga direttamente in una sottostruttura di cartelle.

cd ..

Sale di un livello nella struttura delle cartelle.

find . -iname "*oro*"

Cerca ricorsivamente (in maniera case-insensitive) file o cartelle che contengono "oro" nel nome.

find . -iname "*oro*" -exec mv {} ~/Foresta/Capanna/Cassa \;

Trova i file e li sposta usando il comando mv, uno per uno.

find ~/Giardino/Labirinto -name "*argento*"

Cerca ricorsivamente file che contengono "argento" nel nome (case-sensitive) nel percorso dato.

find ~/Giardino/Labirinto -name "*moneta*"

Ricerca file con "moneta" nel nome nella directory e sottodirectory specificate.

gsh check

Comando di GameShell che controlla se la missione è stata completata correttamente.

head -n 6 Libro_delle_pozioni/pagina_07

Mostra le prime 6 righe del file — utile per ottenere solo titolo e passi.

head -n 6 Libro_delle_pozioni/pagina_13 | tail -n 3

Prende le righe 4–6 (circa) del file per isolare i soli passi della ricetta, ignorando titolo e righe successive.

kill 601438

Termina il processo con PID 601438 usando il segnale predefinito (TERM).

ls

Elenca i file e le cartelle nella directory corrente.

ls -R

Elenca ricorsivamente tutto il contenuto delle sottodirectory.

ls -RA

Come `ls -R`, ma mostra anche i file nascosti (esclusi `.` e `..`).

moneta=~/.Giardino/.../OOOOO_moneta_di_rame_OOOOO

Crea una variabile temporanea contenente un percorso file.

mv \$moneta ~/.Foresta/Capanna/Cassa

Sposta il file indicato dalla variabile `moneta` nella directory di destinazione.

ps

Mostra la lista dei processi attualmente attivi nella shell.

ps | grep Sortilegio

Filtra i processi attivi per mostrare solo quelli che contengono la parola "Sortilegio".

ps | grep Sortilegio | awk '{print \$1}'

Estrae il PID dei processi "Sortilegio" filtrati da `grep`.

ps | grep Sortilegio | awk '{print \$1}' | xargs -r kill -9

Termina con forza tutti i processi "Sortilegio" trovati, usando il segnale `KILL`.

ps | grep Sortilegio | awk '{print \$1}' | xargs -r kill -s TERM

Invia un segnale `TERM` (terminazione gentile) a tutti i processi "Sortilegio" trovati.

tail -n 9 Libro_delle_pozioni/pagina_12

Mostra le ultime 9 righe del file, utile per prendere solo i passi della ricetta evitando il titolo.

Report Missione 31 – La Fata e il Demone Malizioso

Questa missione all’inizio mi ha fatto perdere un sacco di tempo. Sembrava che qualunque cosa facessi, **i sortilegi e i file si rigenerassero** all’infinito. Ho fatto vari tentativi per fermare tutto, ma niente sembrava funzionare davvero. Ci ho provato in diversi modi, cercando di capire bene i collegamenti tra i processi del demone e quelli della fata, ma ogni volta qualcosa tornava a ricrearsi da solo.

Dopo un po’ ho lasciato perdere e ci sono tornato il giorno dopo, più tranquillo. Ho riguardato tutto con più calma, e mi sono accorto che forse stavo facendo troppo. Ho semplicemente controllato bene quali erano i processi **collegati al demone**, senza toccare la fata o i suoi sortilegi, e li ho **interrotti uno alla volta**.

Poi sono andato nella **cartella della Cantina**, ho trovato i file di **carbone** che sporcavano tutto e li ho cancellati.

Alla fine ho controllato con il comando della missione e mi ha dato il messaggio di completamento.

Era più semplice di quanto pensassi. Il segreto era **fermarsi un attimo, guardare bene**, e agire con precisione, senza fare troppo casino. Basta **interrompere i sortilegi giusti e pulire la cantina**. Tutto lì.

```
File Azioni Modifica Visualizza Aiuto
19815_carbone 2848_carbone 38548_carbone 4612_carbone 55368_carbone 7454_carbone
19891_carbone 28666_carbone 38779_carbone 46879_carbone 55976_carbone 7638_carbone
19897_carbone 2887_carbone 38984_carbone 47208_carbone 56026_carbone 8255_carbone
20078_carbone 29130_carbone 39334_carbone 47318_carbone 56034_carbone 8307_carbone
20459_carbone 29283_carbone 39337_carbone 47334_carbone 57008_carbone 8590_carbone
20568_carbone 2957_carbone 39371_carbone 47806_carbone 57202_carbone 8836_carbone
21122_carbone 29796_carbone 3950_carbone 47872_carbone 57336_carbone 8936_carbone
21784_carbone 29853_carbone 39519_carbone 4802_carbone 57339_carbone 9144_carbone
2259_carbone 30302_carbone 39566_carbone 48164_carbone 57504_carbone 9381_carbone
22743_carbone 3047_carbone 40189_carbone 481_carbone 57517_carbone 9590_carbone
23000_carbone 30639_carbone 40373_carbone 48365_carbone 57549_carbone 9890_carbone
23059_carbone 30643_carbone 40657_carbone 48469_carbone 57734_carbone 9928_carbone
23633_carbone 30752_carbone 40719_carbone 48856_carbone 57750_carbone

~/Castello/Cantina
[mission 31] $ rm *_carbone*

~/Castello/Cantina
[mission 31] $ gsh check

Congratulazioni, la missione 31 é stata completata con successo!

[1]- Ucciso "$GSH_TMP/$(gettext "nice_fairy")" (dir: ~)
(dir ora: ~/Castello/Cantina)
Killed
[2]+ Uscita 137 "$GSH_TMP/$(gettext "mischievous_imp")" (dir: ~)
(dir ora: ~/Castello/Cantina)
[ progress was saved in /home/kali/gameshell-save.sh ]
```

Comandi utilizzati:

```
pstree -p $$
kill 2737 2739 2742
cd ~/Castello/Cantina
ls _carbone
rm _carbone
gsh check
```

Report Missione 32 – I Calcoli di Merlino

In questa missione Merlino ha voluto mettermi alla prova con un po' di calcolo mentale. Appena ho lanciato il comando per iniziare, mi ha dato una serie di somme da risolvere una dopo l'altra, senza perdere troppo tempo.

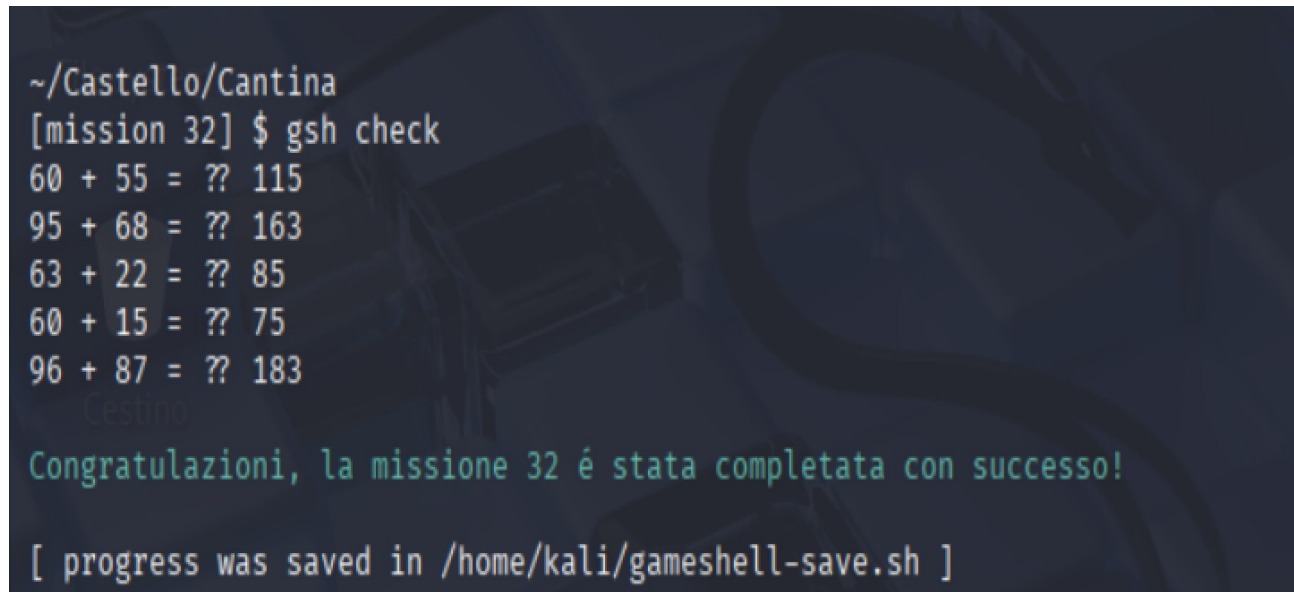
Non è stata difficile, ma serviva un minimo di concentrazione per non sbagliare i conti. Dopo ogni operazione bisognava inserire direttamente il risultato corretto, senza spazi né altro.

Queste sono le somme che mi ha chiesto, con le risposte che ho dato:

- $60 + 55 = 115$
- $95 + 68 = 163$
- $63 + 22 = 85$
- $60 + 15 = 75$
- $96 + 87 = 183$

Una volta risposte tutte in modo corretto, mi è apparso subito il messaggio di completamento della missione.

Missione semplice e veloce, ma ci voleva un attimo di attenzione.



```
~/Castello/Cantina
[mission 32] $ gsh check
60 + 55 = ?? 115
95 + 68 = ?? 163
63 + 22 = ?? 85
60 + 15 = ?? 75
96 + 87 = ?? 183
Cestino
Congratulazioni, la missione 32 è stata completata con successo!
[ progress was saved in /home/kali/gameshell-save.sh ]
```

Report Missione 33 – “Le Moltiplicazioni di Merlino”

All’inizio sembrava impossibile rispondere a tutte quelle moltiplicazioni in meno di 10 secondi. Anche con la mente fresca, il tempo era davvero troppo stretto. Dopo vari tentativi e test, ho iniziato a pensare che ci fosse un trucco nascosto.

Esplorando il Castello, mi sono ricordato del suggerimento nella missione: “Si dice che la libreria contenga alcuni libri di matematica e volumi nascosti.”

Sono andato nella Biblioteca e lì ho trovato due file chiave:

- `Matematica_101`, pieno di numeri (tutti risultati di moltiplicazioni),
- `.Come_imbrogliare_negli_esami`, che diceva chiaramente che quelle **erano le risposte**.

A quel punto ho copiato tutto in un file chiamato `risposte.txt` e ho usato il trucco indicato nella missione (`COMANDO < FILE`) per passare le risposte direttamente a `gsh check`.

Risultato: missione completata in un colpo solo, senza dover scrivere nulla.


```

~/Castello/Palazzo_principale/Biblioteca
[mission 33] $ cd ~/Castello/Cantina

~/Castello/Cantina
[mission 33] $ cp ~/Castello/Palazzo_principale/Biblioteca/Matematica_101 risposte.txt

~/Castello/Cantina
[mission 33] $ gsh check < risposte.txt
44 * 48 = ?? 84 * 90 = ?? 60 * 59 = ?? 81 * 30 = ?? 90 * 35 = ?? 73 * 81 = ?? 5 * 52 = ?? 76 *
9 = ?? 56 * 73 = ?? 92 * 86 = ?? 23 * 17 = ?? 7 * 7 = ?? 13 * 4 = ?? 49 * 39 = ?? 74 * 40 = ??
16 * 48 = ?? 59 * 62 = ?? 98 * 10 = ?? 38 * 13 = ?? 70 * 67 = ?? 62 * 72 = ?? 45 * 5 = ?? 68 *
20 = ?? 15 * 4 = ?? 18 * 13 = ?? 91 * 80 = ?? 36 * 26 = ?? 66 * 37 = ?? 62 * 71 = ?? 79 * 58 =
?? 53 * 98 = ?? 21 * 55 = ?? 6 * 5 = ?? 78 * 68 = ?? 4 * 75 = ?? 45 * 76 = ?? 13 * 20 = ?? 72 *
20 = ?? 18 * 94 = ?? 15 * 59 = ?? 42 * 79 = ?? 53 * 39 = ?? 90 * 67 = ?? 30 * 41 = ?? 57 * 68
= ?? 84 * 14 = ?? 75 * 67 = ?? 24 * 47 = ?? 92 * 56 = ?? 29 * 87 = ?? 90 * 54 = ?? 26 * 82 = ??
80 * 97 = ?? 77 * 98 = ?? 31 * 57 = ?? 92 * 14 = ?? 27 * 76 = ?? 41 * 68 = ?? 25 * 71 = ?? 24
* 64 = ?? 28 * 89 = ?? 47 * 27 = ?? 30 * 9 = ?? 73 * 77 = ?? 35 * 50 = ?? 79 * 89 = ?? 22 * 26
= ?? 42 * 22 = ?? 64 * 56 = ?? 17 * 100 = ?? 17 * 85 = ?? 79 * 47 = ?? 35 * 66 = ?? 89 * 20 = ?
? 50 * 77 = ?? 5 * 31 = ?? 30 * 53 = ?? 70 * 88 = ?? 77 * 9 = ?? 9 * 20 = ?? 84 * 47 = ?? 89 *
13 = ?? 30 * 21 = ?? 19 * 2 = ?? 83 * 5 = ?? 98 * 77 = ?? 85 * 57 = ?? 45 * 7 = ?? 25 * 9 = ??
57 * 92 = ?? 35 * 83 = ?? 13 * 3 = ?? 99 * 22 = ?? 54 * 20 = ?? 35 * 4 = ?? 9 * 22 = ?? 57 * 81
= ?? 91 * 31 = ?? 99 * 17 = ?? 16 * 51 = ??
Congratulazioni, la missione 33 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
Mentre stai aspettando, vedi un pesce nuotare in cerchio...

```

Comandi utilizzati

```

cd ~/Castello/Palazzo_principale/Biblioteca
cat Matematica_101
cat .Come_imbrogliare_negli_esami
cd ~/Castello/Cantina
cp ~/Castello/Palazzo_principale/Biblioteca/Matematica_101 risposte.txt
gsh check < risposte.txt

```

Missione 34 – “L’inventario dei Grimori”

In questa missione dovevo trovare tutti i libri di magia e salvarli in un file chiamato `inventario.txt`. Dopo aver esplorato l’Ufficio di Merlino nella Biblioteca, ho trovato una lunga lista di file chiamati `grimorio_` seguiti da numeri.

Capito che erano loro i libri magici, li ho selezionati tutti usando un comando per creare l’elenco automatico, e l’ho salvato nel cassetto come richiesto. Alla fine, ho controllato il contenuto del file e, appena sicuro che c’erano solo i grimori, ho lanciato il comando `gsh check` e la missione è stata completata senza problemi.

```
~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino
[mission 34] $ gsh check

Congratulazioni, la missione 34 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
Mentre tu stai aspettando, un pipistrello vola via...

|                                     |
--+-----+-----+-----+-----+
| Usa il comando                    |
|   $ gsh help                     |
| per ottenere una lista dei comandi "gsh". |
--+-----+-----+-----+-----+
|                                     |
```

Comandi utilizzati

```
cd ~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino
ls grimorio_* > Cassetto/inventario.txt
less Cassetto/inventario.txt
gsh check
```

Missione 35 – L'Archivio di Merlino

Obiettivo

Dovevo trovare i libri nell'ufficio di Merlino che contenevano l'elemento alchemico *gsh* (Glutatione). Poteva essere scritto in qualunque modo, con maiuscole o minuscole, e bisognava ignorare quelli sotto chiave. Non dovevo far comparire la parola "gsh" nei risultati, solo la lista dei titoli.

Svolgimento

Per prima cosa ho dato un'occhiata agli obiettivi con `gsh goal`.

Ho esplorato la struttura della directory, partendo dal livello iniziale, per cercare i libri.

Nel percorso `biblioteca/merlino/libri` ho lanciato un `grep` con le opzioni giuste per trovare i file che contenevano "gsh", ignorando maiuscole/minuscole, e ho salvato i risultati in un file, eliminando gli errori.

Poi ho provato a entrare nell'archivio direttamente, ma non esisteva.

Allora sono passato dal Castello: ho attraversato il Palazzo principale e sono arrivato alla Biblioteca.

Dentro ho trovato l'Ufficio di Merlino, dove erano conservati un sacco di grimori.

Lì ho cercato di nuovo "gsh" nei file, sempre senza visualizzare i contenuti.

Una volta ottenuta la lista dei grimori giusti, ho controllato con `gsh check`.

Missione completata con successo.

```
grimorio_daqiHWkcBl
grimorio_DIGsXbrzmVfMGL0zQMxPjhhHNLWHSxo
grimorio_FyMaNtqgGLAjsyGaXlJFQZ
grimorio_gjLSqciJevlxSLq
grimorio_hFKziVUjiTxbzuLMiEx
grimorio_HFllKhxRLoMvYwR
grimorio_hOSDGIrGUBZgkskxVbBPKJp
grimorio_jGfCAllsrMmnN
grimorio_lOJeLHgJQvaIgfQ
grimorio_mDdiltuvymuBgEVVVYpjOzxjCJ
grimorio_MreMTICNgNRGmiPOQb
grimorio_MVRSFbAlwQEnyVPHIiFfuskB
grimorio_NwXIxLgzKYwlQvWQAoiHG
grimorio_NztjPEFsOAdBYMEbsKBSmcgItmKkYt
grimorio_QPslkVNypBCxHZXqwCoOpHegHSFmT
grimorio_rkWCQQTGuRYmkYPgtPHVcQUyFUqVl
grimorio_RpRelAgAyxPFuxjZJZAcNni
grimorio_SRGWeOQMICB
grimorio_tzGpzLwLHKzxXbWTPXJTvrRd
grimorio_viehimiFssbRaPfQjn
grimorio_XKHwxvMRVo
grimorio_YAVwlnrvogsgqpWZAwn
grimorio_YKFaxqvwfGs
grimorio_zXjjASpbxjRFXFlnEURSXRw

~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino
[mission 35] $ gsh check

Congratulazioni, la missione 35 è stata completata con successo!
```

Comandi utilizzati

```
gsh goal
la
grep -il 'gsh' biblioteca/merlino/libri/* 2> /dev/null > lista_libri.txt
cd Castello
cd Palazzo_principale
cd Biblioteca
cd Ufficio_di_Merlino
grep -l -i gsh grimorio* 2>/dev/null
gsh check
```

Missione 36

Merlino, ormai pazzo, cammina per la torre dell'osservatorio borbottando frasi incomprensibili. Il compito della missione è quello di decifrare le sue divagazioni e trovare la chiave segreta che solo lui conosce. La difficoltà sta nel fatto che il suo messaggio è pieno di errori, e quindi il nostro obiettivo diventa quello di isolare e raccogliere la chiave segreta.

Un importante suggerimento che ci viene dato è di utilizzare un reindirizzamento a file per completare la missione. Quindi, ci occorrerà sfruttare i comandi di redirectione per estrarre correttamente l'informazione.

Svolgimento della Missione

Raccolta dell'output di errore di Merlino: Il primo passo che faccio è eseguire il programma di Merlino, che, nonostante sia pazzo, deve restituire una sorta di informazione utile (ovvero la chiave). Tuttavia, il programma restituirà sicuramente degli errori, che dovrò

raccogliere. Per fare questo, utilizzo il comando:

```
./merlin 2> merlinstd.txt
```

Questo comando esegue `merlin` e reindirizza l'output di errore (`stderr`) in un file chiamato `merlinstd.txt`. Questo è importante perché la chiave che cerchiamo probabilmente si trova proprio in questi errori, non nell'output standard.

Visualizzazione del contenuto del file di errore: Dopo aver eseguito il comando precedente, posso vedere il contenuto del file `merlinstd.txt` per verificare se effettivamente contiene la chiave segreta. Per farlo, uso il comando `cat`:

```
cat merlinstd.txt
```

Il contenuto del file mostra una lunga stringa alfanumerica:

```
LACHIAVESEGRETASITROVAINSTDERR
```

Questa è la chiave che stavo cercando, nascosta nell'output di errore di Merlino!

Verifica della chiave con il comando `gsh check`: Ora che ho la chiave, devo usarla per completare la missione. Per farlo, devo passarla come input al comando `gsh check`. Inizialmente, ho provato a usare il comando errato (`chek`), ma ho ricevuto un errore. Dopo aver corretto il comando, uso:

```
gsh check < merlinstd.txt
```

Il comando `gsh check` prende la chiave dal file `merlinstd.txt` e la verifica.

```
~/Castello/Osservatorio
[mission 36] $ ls mappa_del_cielo
mappa_del_cielo

~/Castello/Osservatorio
[mission 36] $ gsh check < merlin2std.txt
bash: merlin2std.txt: File o directory non esistente

~/Castello/Osservatorio
[mission 36] $ LACHIAVESEGRETASITROVAINSTDERR
LACHIAVESEGRETASITROVAINSTDERR: comando non trovato

~/Castello/Osservatorio
[mission 36] $ ./merlin 2> merlinstd.txt
LACHIAVESEGRETASITROVAINSTDERR

~/Castello/Osservatorio
[mission 36] $ cat merlinstd.txt
AswyPUxyfcJAhbuIBwDiDP0xzhUUaLKsECJB8mQBdfWuWJxRVFYEEoSnbYUApRYMasmgmqiFMUPQrjDdrKVOQCpdSdbhlE
TKqDKqZGExtGMeuDePeemGabBOosvxOHpXNEUUGttCQgKJRCbDFMSHiABzBaBMDmghlhHpljIACdwNGFulmahaidxOnayL
kgWWxiQEzb

~/Castello/Osservatorio
[mission 36] $ gsh check < merlinstd.txt
Qual è la chiave corretta?
Congratulazioni, la missione 36 è stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```


Comandi Utilizzati

```
./merlin 2> merlinstd.txt
```

Esegue il programma Merlino e reindirizza gli errori in un file, dove posso poi recuperare la chiave segreta.

```
cat merlinstd.txt
```

Visualizza il contenuto del file con gli errori per identificare la chiave segreta.

```
gsh check < merlinstd.txt
```

Verifica la chiave segreta usando il comando `gsh check`.

Missione 37: La Porta per il Quartiere dei Re

Obiettivo della missione:

La porta per il quartiere dei Re si trova nella sala del trono. Devi riuscire a entrare nel quartiere dei Re, ma l'accesso potrebbe essere limitato.

Passaggi seguiti:

Partenza dalla posizione iniziale:

Ho iniziato dalla directory `~/Castello/Osservatorio`.

Navigazione nel castello:

Ho usato il comando `cd ..` per uscire dall'**Osservatorio** e ho visualizzato i contenuti della directory con `ls`, trovando diverse opzioni, tra cui **Cantina**, **Grande_sala**, **Palazzo_principale**, e **Torre_principale**.

Esplorazione della Grande Sala:

Ho deciso di esplorare la **Grande_sala**, usando `cd Grande_sala` per entrare nella cartella e successivamente visualizzare i contenuti con `ls`. Ho trovato diversi arazzi e oggetti decorativi, ma non mi servivano al momento, quindi sono uscito dalla sala usando `cd ...`.

Accedere alla Sala del Trono:

Sono passato al **Palazzo_principale** con `cd Palazzo_principale`, poi sono entrato nella cartella **Sala_del_trono** con `cd Sala_del_trono`. Qui ho usato `ls -l` per verificare i permessi e i metadati dei file, trovando la directory **Quartiere_dei_Re**.

Modifica dei permessi:

Poiché l'accesso alla directory **Quartiere_dei_Re** era limitato, ho provato a usare `chmod +x Quartiere_dei_Re/` per modificare i permessi e permettermi di accedere, ma inizialmente ho ricevuto un errore. Dopo aver verificato, ho eseguito correttamente `chmod +x Quartiere_dei_Re/` e finalmente ho avuto accesso alla cartella.

Entrata nel Quartiere dei Re:

A questo punto, ho usato `cd Quartiere_dei_Re/` per entrare nella directory e completare la missione.

Verifica della missione:

Infine, ho utilizzato `gsh check` per verificare se la missione era stata completata correttamente. La missione è stata completata con successo.

```
[mission 37] $ chmod +x Quartiere_dei_RE/
chmod: impossibile accedere a 'Quartiere_dei_RE/': File o directory non esistente

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ chmod +x Quartiere_dei_RE
chmod: impossibile accedere a 'Quartiere_dei_RE': File o directory non esistente

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ chmod +x Quartiere_dei_RE
chmod: impossibile accedere a 'Quartiere_dei_RE': File o directory non esistente

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ cd Quartiere_dei_Re
bash: cd: Quartiere_dei_Re: Permessi negati

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ chmod +x Quartiere_dei_RE/
chmod: impossibile accedere a 'Quartiere_dei_RE/': File o directory non esistente

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ chmod +x Quartiere_dei_Re/

~/Castello/Palazzo_principale/Sala_del_trono
[mission 37] $ cd Quartiere_dei_Re/

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 37] $ gsh check

Congratulazioni, la missione 37 è stata completata con successo!
```

Comandi Utilizzati:

1. `cd ..`
2. `ls`
3. `cd Grande_sala`
4. `ls`
5. `cd ..`
6. `cd Palazzo_principale`
7. `ls`
8. `cd Sala_del_trono`
9. `ls -l`
10. `chmod +x Quartiere_dei_Re/`
11. `cd Quartiere_dei_Re/`
12. `gsh check`

Missione 38

Obiettivo:

Scoprire la combinazione per aprire la cassaforte del Re. La combinazione è scritta su un biglietto nella sua stanza, ma il Re ha cercato di renderla illeggibile.

Fasi svolte:

Esplorazione della directory:

Ho usato il comando `ls -la` per vedere i file nella directory `Quartiere_dei_Re` e ho trovato i seguenti file:

biglietto

.biglietto_segreto

Accesso al file nascosto:

- Ho provato a leggere **.biglietto_segreto** con il comando `cat`, ma il sistema ha dato "Permesso negato".
- Ho quindi cambiato i permessi del file con il comando `chmod +r .biglietto_segreto` per poterlo leggere.

Lettura del contenuto:

- Dopo aver cambiato i permessi, ho eseguito il comando `cat` `.biglietto_segreto` e ho trovato la combinazione **6175143317**.

```
~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $ ls -la
totale 16
drwxrwxr-x 2 kali kali 4096 23 apr 07.34 ./
drwxrwxr-x 3 kali kali 4096 18 apr 17.49 ../
-rw-rw-r-- 1 kali kali  11 23 apr 07.34 biglietto
--w--w--- 1 kali kali  11 23 apr 07.34 .biglietto_segreto

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $ cat .biglietto_segreto
cat: .biglietto_segreto: Permesso negato

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $ chmod +r .biglietto_segreto

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $ cat .biglietto_segreto
6175143317

~/Castello/Palazzo_principale/Sala_del_trono/Quartiere_dei_Re
[mission 38] $ gsh check
Qual'è la combinazione per aprire la cassaforte del Re? 6175143317

Congratulazioni, la missione 38 è stata completata con successo!
```

Comandi utilizzati:

- `ls -la`
- `cat .biglietto_segreto`
- `chmod +r .biglietto_segreto`

Risultato: La combinazione per aprire la cassaforte del Re è **6175143317**.

Missione 39

Obiettivo:

Il compito era rubare la **corona del Re** dalla cassaforte nella **Sala del Trono** e trasportarla nella **Cassa** situata nella **Capanna della Foresta**. La base della **corona** conteneva anche una serie magica di numeri che dovevano essere osservati e ricordati.

Fasi della Missione:

Accesso alla Cassaforte:

Prima di tutto, sono arrivato alla **Cassaforte** nella **Sala del Trono** e, utilizzando il comando `ls -la`, ho notato che i permessi sulla **corona** non mi permettevano di accedervi.

Per risolvere questo, ho dato il comando `chmod +rwx Cassaforte/` per ottenere i permessi necessari per leggere e spostare il file.

Osservazione della Corona:

- Una volta modificati i permessi, ho usato il comando `cat corona` per leggere il contenuto della **corona** e scoprire che alla base era incisa la sequenza **397**, una combinazione magica che mi è stata chiesta più avanti.

Spostamento della Corona:

- Il passo successivo era spostare la **corona** nella **Cassa** della **Capanna** nella **Foresta**. Tuttavia, quando ho provato a eseguire il comando `mv`, mi sono imbattuto in errori perché la destinazione non esisteva.
- Ho quindi navigato tra le directory, utilizzando i comandi `cd` e `ls`, per accertarmi che la **Cassa** nella **Capanna** fosse pronta a ricevere la **corona**.

Trasferimento con Successo:

Dopo aver verificato, sono riuscito a spostare correttamente la **corona** dalla **Cassaforte** alla **Cassa** con il comando `mv /home/kali/gameshell1.4/World/Castello/Palazzo_principale/Sala_del_trono/Cassaforte/corona /home/kali/gameshell1.4/World/Foresta/Capanna/Cassa`.

Completamento della Missione:

Infine, una volta messa al sicuro la **corona**, mi è stata chiesta la sequenza numerica incisa alla base. La risposta corretta era **397**, che ho inserito con successo completando la missione.


```
~/Foresta/Capanna
[mission 39] $ ls
Cassa/

~/Foresta/Capanna
[mission 39] $ cd Cassa

~/Foresta/Capanna/Cassa
[mission 39] $ ls
26564_arazzo_09  40151_arazzo_06  dipinto_RwwLrXUd  MonetA_dOr0_2      stendardo_2
30898_arazzo_03  48894_arazzo_02  moneta_1          00000_moneta_dargento_00000  stendardo_3
32466_arazzo_04  49124_arazzo_05  moneta_2          00000_moneta_di_rame_00000  stendardo_4
3834_arazzo_01   630_arazzo_08    moneta_3          registro.txt        storico.txt
4003_arazzo_10   6515_arazzo_07   moneta_doro_1     stendardo_1

~/Foresta/Capanna/Cassa
[mission 39] $ pwd
/home/kali/gameshell.4/World/Foresta/Capanna/Cassa

~/Foresta/Capanna/Cassa
[mission 39] $ mv /home/kali/gameshell.4/World/Castello/Palazzo_principale/Sala_del_trono/Cassa
forte/corona /home/kali/gameshell.4/World/Foresta/Capanna/Cassa

~/Foresta/Capanna/Cassa
[mission 39] $ gsh check
Quali sono le 3 cifre incise alla base della corona? 397

Congratulazioni, la missione 39 è stata completata con successo!
```

Comandi Utilizzati:

- `ls -la`: Elenco dei file nella directory per verificarne i permessi.
- `chmod +rwx`: Modifica dei permessi di accesso per poter leggere e spostare la **corona**.
- `cat corona`: Lettura del contenuto della **corona** per scoprire la sequenza magica.
- `mv`: Spostamento della **corona** dalla **Cassaforte** alla **Cassa**.

La missione è stata completata con successo. La **corona** è ora al sicuro nella **Cassa** della **Capanna**, e la combinazione magica **397** è stata registrata. Il progresso è stato salvato.

Missione 40 – Il Rubino nel Labirinto

All'inizio della missione mi trovavo già nella **Cassa**, all'interno della **Capanna nella Foresta** (`~/Foresta/Capanna/Cassa`). L'obiettivo era chiaro: trovare un file nascosto nel **labirinto del Giardino**, contenente il **rubino**, e portarlo qui nella Cassa.

Per prima cosa ho iniziato a risalire la struttura delle directory. In un primo tentativo ho scritto `cd .`, dimenticando lo spazio, e il terminale giustamente mi ha restituito un errore. Corretto l'errore con `cd ..`, ho eseguito il comando un paio di volte per tornare nella **directory principale** (`~`), dove ho potuto vedere tutte le aree del mondo: Castello, Foresta, Giardino, Montagna, Stalla, oltre ad alcuni file come `lista_libri.txt`.

Sapendo che il rubino si trovava nel **Labirinto**, mi sono diretto nella directory Giardino, poi dentro Labirinto. Una volta lì, ho dato un'occhiata al contenuto usando `ls`: ho trovato una lunga lista di cartelle con nomi criptici, probabilmente generate automaticamente.

A quel punto ho capito che cercare il file giusto manualmente sarebbe stato poco pratico. Ho quindi utilizzato il comando:

```
find -type f
```

per ottenere un elenco **solo dei file**, ignorando tutte le cartelle. Il risultato è stato una lista di sei file con percorsi piuttosto profondi e contorti.

A questo punto, ho deciso di ispezionare il contenuto di ciascuno di questi file usando `cat`, cercando la parola chiave **rubino**, visto che il nome del file non aiutava affatto. I primi file contenevano semplicemente delle **pietre** con dei codici accanto — nulla che avesse a che fare con il nostro obiettivo.

Poi, finalmente, in uno dei file ho trovato questo:

```
63122 rubino ef178b44b9ee6fc1d8ed7d2b4cc241d48dcdcf220
```

Era proprio lui: il **rubino**.

Il file si trovava nel percorso:

```
./73ae28a35dc0e1a44fc2c56b1d3/1f4f88b92e24/d878e8354fd/63122
```

A questo punto ho usato `mv` per spostarlo direttamente nella Cassa nella Capanna, senza passaggi intermedi:

```
mv ./73ae28a35dc0e1a44fc2.../63122 ~/Foresta/Capanna/Cassa
```

```
[mission 40] $ cat ./941f199b5d4ca3866fd43b/28c955cc38431e401e055ffd8f20d/46aa50547441562413dc10f4/36587
36587 pietra 73d7bf8ec6087609c03a496286fbc55854db597

~/Giardino/Labirinto
[mission 40] $ cat ./57817128da5b412926/1222cfb912f70378f61/12ffc43f2379ff47c/46761
46761 pietra 377d18ef5ea13921a3f557f776c3571535dabd5c

~/Giardino/Labirinto
[mission 40] $ cat ./e57a8cacc774898e791fa96331c4cb6/d0fa11ecfab8dbce7b596f8352/053a594b0df4be4e2a7da7a0a7dcf8/3306
3306 pietra df23fd86f678de3b00fd9c501d9497c36b8085

~/Giardino/Labirinto
[mission 40] $ cat ./73bbce283/312caab5036028ac7fae718ba7ba6e/d3d5480445327/23813
23813 pietra 68732c5ba939fd3da9846170ad3a4e69ac033280

~/Giardino/Labirinto
[mission 40] $ cat ./73ae28a35dc0e1a44fc2c56b1d3/1f4f88b92e24/d878e8354fd/63122
63122 rubino ef178b44b9ee6fc1d8ed7d2b4cc241d48dcdcf220

~/Giardino/Labirinto
[mission 40] $ mv ./73ae28a35dc0e1a44fc2c56b1d3/1f4f88b92e24/d878e8354fd/63122 ~/Foresta/Capanna/Cassa

~/Giardino/Labirinto
[mission 40] $ gsh check

Congratulazioni, la missione 40 è stata completata con successo!
```

Comandi utilizzati

```
cd ..
cd ..
cd ..
ls
cd Giardino
ls
cd Labirinto
ls
find -type f
cat ./941f.../36587
cat ./5781.../46761
cat ./e57a.../3306
cat ./73bb.../23813
cat ./73ae.../63122
mv ./73ae.../63122 ~/Foresta/Capanna/Cassa
gsh check
```

Missione 41: Trova e sposta il diamante nella tua cassa o meglio...il “Diamante”:-)

Obiettivo della missione:

La missione 41 prevedeva una semplice attività: trovare un file nel **Labirinto** contenente la stringa "diamante" e spostarlo nella tua cassa. Un compito abbastanza facile, o almeno così sembrava... fino a quando il gioco non ha deciso di darmi una piccola sorpresa.

Il Piano di Attacco (Spoiler: Non ha funzionato)

Primo passo: Navigare nel Labirinto

Partiamo dal principio, come da copione. Mi sono diretto nel **Labirinto** dove tutti i file si nascondono come se fosse un gioco di nascondino. Ho dato il classico comando per entrare:

```
cd ~/Giardino/Labirinto
```

Tentativi di ricerca automatica

Dopo aver preso un bel respiro, ho pensato: "Ok, nulla di complicato. Usa il buon vecchio comando `find` per cercare tutti i file e poi `grep` per cercare la parola 'diamante'. Facile, no?".

Mi ero preparato ad un esito positivo e veloce, ma... sorpresa! Non è andata così.

Ho iniziato con il comando classico per cercare la stringa "diamante":

```
find . -type f -print0 | xargs -0 grep -l "diamante"
```

Ma niente! Nessun risultato. Forse un errore mio, penso. Riprovo con altri comandi, anche con delle piccole variazioni:

```
find . -type f | xargs grep "diamante"
find . -type f | xargs grep "diamante"
find . -type f | xargs grep "_diamante"
```

Ancora niente! E mentre il mio schermo si riempiva di file, io continuavo a chiedermi:

"Cosa sta succedendo? Ho sbagliato qualcosa?"

1. La Grande Rivelazione: "Diamonte" e non "Diamante"

Poi, finalmente, quando gli occhi stavano ormai incrociandosi per la frustrazione, ho capito la verità: il gioco non aveva scritto **"diamante"**, ma **"diamonte"**! Sì, avete letto bene. La differenza di una sola lettera ha mandato tutto in tilt.

Era praticamente come cercare una pepita d'oro nel fango, solo che il fango era composto da file che non avevano niente a che fare con quello che cercavo. E il diamante... o meglio, il "diamonte", si nascondeva là fuori, nell'ombra, a sfidarmi.

2. La Ricerca Manuale (ma che stanchezza)

Dopo aver capito il mio errore, ho dovuto avviare una ricerca manuale. Ho esplorato i file uno per uno, in cerca di quel benedetto file che conteneva la stringa **"diamonte"**. E finalmente, dopo aver frugato tra centinaia di file, eccolo:

```
./3080c434c36f8c680312/6569cd9df84deacf09/cb9395aee
```

Dentro, una piccola rivelazione che mi ha fatto sorridere amaramente:

```
cb9395aee diamonte 83a4433fcd5214643e23ea213a930c2acbac2b4e
```

Diamonte. Non ci crederete, ma era davvero scritto così!

E qui, tra un sospiro e l'altro, ho capito che la ricerca automatica non avrebbe mai funzionato, non con quella "e" in più.

Spostamento del File nella Cassa

A quel punto, ho fatto ciò che ogni bravo avventuriero avrebbe fatto: ho spostato il file nella mia cassa. Per fortuna, il comando di spostamento era più semplice di una ricerca, quindi ho eseguito:

```
mv ./3080c434c36f8c680312/6569cd9df84deacf09/cb9395aee  
~/Foresta/Capanna/Cassa
```

Verifica del Successo

Dopo aver finalmente messo le mani sul "diamante" (o "diamonte", a seconda di come vogliamo chiamarlo), ho verificato il completamento della missione con il comando finale:

```
gsh check
```

Ed ecco la risposta che stavo aspettando:

```
Congratulazioni, la missione 41 è stata completata con successo!
```

Conclusioni:

Alla fine, la missione è stata completata, ma non senza un buon po' di frustrazione. La ricerca automatica non ha funzionato semplicemente perché **la parola chiave non era "diamante" ma "diamonte"**. Quindi, dopo aver sbagliato decine di comandi, ho dovuto fare tutto manualmente, con un po' di pazienza e molta ironia. Spero che il gioco, la prossima volta, decida di non "cambiare" le parole a caso.


```
File Azioni Modifica Visualizza Aiuto
228533554377ae97a26      89ea6b65ffe091ce6340537b76b3ea  a32ef2fe870c586c58

./cddcb00d157698aa4dccc12/e0932843367a1a22b4d148a0cc973b6:
15cb9ac6fdb4459804      2790dc47692502      8396247c1b371b8ea76961f93b
1642d44b057d99005a5ea812474c  4b7c982d93392752b054723c8  cde5fbd5da3cc28b22068c7
1750864cef5179f4d34cec202540e33  5745e0725de6c6
248fbfcbc0131e5a3a9c0089b25      6357dabc9c70b49a3654392c1

./cddcb00d157698aa4dccc12/eb11202cfdb159065396632d64756:
0b788e446704a53c      7a250ff0211815ab8a3c28      c02a86616b783b8ea0a2f4  f0717b7f
237e9ac0      b2e8a9d0eacc3e3f801101b4d  c7a57f407229da5fb07
4546888d28d8d8b60a69  bd6b735fcf2d82549c0762c  df478f9a81e9906

./cddcb00d157698aa4dccc12/ef30547ec29f987c87f07d4c2:
42a209e2357050fe863a0      63426493104a7      c22d23d23c89b
45461fd7bed259fab5b3bb6846a  770e0c840122438a2d6a7763f58d2d  e39df68d9e3bb3363ae26
5b5cae2cf78bf596      77dcf1952a401
5c5a32750f5519d      b9136181d45e1fee86ba401b5

~/Giardino/Labirinto
[mission 41] $ mv ./3080c434c36f8c680312/6569cd9df84deacf09/cb9395aee ~/Foresta/Capanna/Cassa

~/Giardino/Labirinto
[mission 41] $ gsh check

Congratulazioni, la missione 41 é stata completata con successo!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

Comandi Utilizzati:

```
cd ~/Giardino/Labirinto
find . -type f -print0 | xargs -0 grep -l "diamante"
find . -type f | xargs grep "diamante"
find . -type f | xargs grep "diamante"
find . -type f | xargs grep "_diamante"
mv ./3080c434c36f8c680312/6569cd9df84deacf09/cb9395aee ~/Foresta/Capanna/Cassa
gsh check
```

Missione 42 – Quanto deve il Re?

All'inizio sembrava una missione come le altre, ma alla fine serviva attenzione a ogni dettaglio. L'obiettivo era capire **quanto deve ancora il Re**, cioè trovare tutte le sue spese che **non ha pagato**, e sommarle.

Individuare il file giusto

Ho prima cercato i file che contenevano la parola "pergamena" nel nome:

Comando usato:

```
ls _p_e_r_g_a_m_e_n_a
```

E da lì ho trovato un file con un nome tipo:

```
23943ae0df41dd_p_e_r_g_a_m_e_n_a_23943ae0df41dd63
```

Visualizzare solo le spese del Re

Una volta trovato il file, volevo vedere **solo le righe in cui il Re ha comprato qualcosa**, quindi ho fatto:

Comando usato:

```
grep "il Re" 23943ae0df41dd_p_e_r_g_a_m_e_n_a_23943ae0df41dd63
```

Così ho ottenuto l'elenco completo delle sue spese, ma includeva anche quelle già pagate. Passaggio 3 – Filtrare solo le spese non pagate

Poi ho tolto tutte le righe dove c'era scritto "PAGATO", per vedere solo quelle ancora da saldare:

Comando usato:

```
grep "il Re" 23943ae0df41dd_p_e_r_g_a_m_e_n_a_23943ae0df41dd63 | grep -v "PAGATO"
```

Mi ha mostrato le spese fatte dal Re che però **non risultavano pagate**.

Ottimizzazione con un solo comando

Alla fine però ho voluto fare tutto in **un solo comando**, per superare la missione in modo pulito, visto che GameShell conta i comandi usati. Quindi ho usato una riga sola per fare tutto: cercare, filtrare, prendere i numeri e sommare.

Comando finale utilizzato:

```
cat * | grep "Re" | grep -v "PAGATO" | grep -o '[0-9]+' | awk '{s+=$1} END {print s}'
```

Questo comando fa esattamente quello che serviva:

- Prende tutte le righe dove compare il Re
- Toglie quelle già pagate
- Estrae solo i numeri (cioè i costi)
- Somma tutto e stampa il risultato

Il totale era 21. Appena l'ho scritto dentro gsh check, il sistema mi ha confermato che era giusto:

```
(0)
~/Stalla
[mission 42] $ cat * | grep "Re" | grep -v "PAGATO"
Rebecca Leone ha comprato una mucca per 4 di rame.
il Re ha comprato una mucca per 6 di rame.
il Re ha comprato una palla di cuoio per 5 di rame.
il Re ha comprato uno sgabello per 2 di rame.
il Re ha comprato una borsa in pelle per 5 di rame.
il Re ha comprato un cavallo per 3 di rame.
(1)
~/Stalla
[mission 42] $ gsh check
Quanto possiede il re? 21

Congratulazioni, la missione 42 è stata completata con successo!
```

Missione 43 – Obiettivo: trovare il numero esatto di articoli comprati ma non pagati, usando un solo comando.

Dopo alcuni tentativi andati male (tipo contare tutto quello che non conteneva la parola "Pagato", col risultato che mi sparava fuori novemila e passa righe), ho capito che dovevo ragionare in modo più preciso. Non si trattava solo di escludere i pagamenti, ma di includere **solo gli acquisti**, e poi tra quelli togliere quelli già pagati.

Alla fine, il comando giusto — l'unico che ha funzionato — è stato:

```
grep "ha comprato" p_e_r_g_a_m_e_n_a | grep -v "PAGATO" | wc -l
```

Vediamo **perché funziona**, passo per passo:

1. **grep "ha comprato" *_p_e_r_g_a_m_e_n_a_***

Questo serve per cercare, in tutti i file che hanno nel nome "pergamena", solo le righe che contengono la frase **"ha comprato"**. In pratica, isola **tutti gli acquisti**, cioè tutte le frasi dove qualcuno compra qualcosa.

2. **| grep -v "PAGATO"**

A questo punto, con la pipe, prendo quelle righe e le passo al secondo **grep**, che **esclude tutte le righe** che contengono "PAGATO". Quindi rimangono solo gli articoli **acquistati ma non pagati**.

3. **| wc -l**

Infine, uso **wc -l** per **contare le righe risultanti**. Ogni riga è un oggetto non pagato, quindi il numero finale è esattamente il numero di articoli che stavamo cercando.

Il risultato, in questo caso, è stato **57**,

```
|  
Cestino  
(0)  
~/Stalla  
[mission 43] $ grep "ha comprato" *_p_e_r_g_a_m_e_n_a_* | grep -v "PAGATO" | wc -l  
57  
(1)  
~/Stalla  
[mission 43] $ gsh check  
Quanti articoli non pagati hai? 57  
  
Congratulazioni, la missione 43 è stata completata con successo!
```

Missione 44 – Decifrare il messaggio segreto

Obiettivo della missione

La missione consisteva nel recuperare un messaggio segreto nascosto nel cassetto dell'ufficio di Merlino. Il messaggio era criptato con il **cifrario di Cesare**, una tecnica di cifratura che sposta le lettere dell'alfabeto di un certo numero di posizioni. Il nostro compito era scoprire di quale spostamento si trattasse e decifrare il messaggio.

Passaggi eseguiti

1. **Accedere al cassetto di Merlino** Ho iniziato la missione spostandomi nella directory corretta. La cartella da raggiungere si trovava sotto il percorso `~/Castello/Palazzo_principale/Biblioteca/Ufficio_di_Merlino/Cassetto`. Per farlo, ho utilizzato il comando `cd`, che permette di navigare tra le cartelle, fino ad arrivare nella cartella contenente il messaggio segreto.
2. **Esplorare i contenuti del cassetto** Una volta arrivato nel cassetto, ho elencato i file presenti nella cartella con il comando `ls`. Tra questi, ho trovato il file **messaggio_segreto**, che era il candidato giusto per il nostro obiettivo. Per visualizzare il contenuto di questo file, ho utilizzato il comando `cat`. Questo comando stampa a schermo il contenuto del file, che si è rivelato cifrato come segue:

```
dhrfgr fbab yr zvr ibybagá:  
nienv yn zvn pnffn r ghggd dhryyb pur pbagvvar.  
dhrfgn pnffn é aryyn pnagvan r yn cnebyn cre sneyn  
nccnever é: rdfq  
zreyvab yb fgertbar
```

3. **Decifrare il messaggio** Dopo aver visualizzato il testo criptato, era necessario decifrarlo. Il messaggio sembrava essere cifrato con il cifrario di Cesare, e il suggerimento indicava uno spostamento di **13** posizioni nell'alfabeto. Per decifrare il messaggio, ho utilizzato il comando `tr`, che è utile per sostituire i caratteri di una stringa con altri caratteri. Il comando `tr "a-z" "n-za-m"` effettua proprio una sostituzione di questo tipo, dove ogni lettera viene sostituita con la lettera che si trova 13 posizioni più avanti nell'alfabeto (se si considera l'alfabeto come una sequenza ciclica).

Questo comando ha trasformato il messaggio criptato in un testo leggibile:

```
queste sono le mie volontà:  
avrai la mia cassa e tutto quello che contiene.  
questa cassa é nella cantina e la parola per farla  
apparire é: eqsd  
merlino lo stregone
```

4. La parte fondamentale del messaggio era la parola chiave: **"eqsd"**, che è la risposta alla missione.
5. **Fornire la chiave** Una volta ottenuta la parola chiave **"eqsd"**, ho dovuto fornirla per completare la missione. Ho utilizzato il comando `gsh check`, che richiedeva di inserire la chiave che avrebbe fatto apparire la cassa di Merlino. Rispondendo con **"eqsd"**, ho ricevuto la conferma che la missione era stata completata con successo.

Conclusioni sui comandi utilizzati:

- **cd**: questo comando è stato usato per navigare tra le directory, raggiungendo la cartella contenente il file segreto.
- **ls**: è stato utilizzato per esplorare il contenuto della cartella e individuare il file da decifrare.
- **cat**: permette di visualizzare il contenuto di un file, che in questo caso conteneva il messaggio criptato.
- **tr**: è stato utilizzato per applicare il cifrario di Cesare. Il comando `tr "a-z" "n-za-m"` sposta ogni lettera di 13 posizioni nell'alfabeto, decifrando il messaggio.
- **gsh check**: infine, ho usato questo comando per fornire la chiave e completare la missione con successo.

Conclusione di GameShell

Giocando a GameShell ho avuto modo di confrontarmi con una serie di sfide che mi hanno messo alla prova non solo a livello tecnico, ma anche a livello di pazienza e strategia. Ogni missione ha richiesto di usare comandi come `grep`, `tr`, `cat`, `ls`, `wc`, `find`, `chmod`, `rm`, `cp`, `mv`, `tar`, `echo`, `touch`, `nano`, `head`, `tail`, `sort`, `diff`, `cut`, `ps` e `df` in modo sempre più mirato, spingendomi a capire a fondo come funzionano gli strumenti da terminale. Non sono stati semplici passaggi da seguire, ma piuttosto piccoli enigmi che mi hanno obbligato a ragionare su come combinare i comandi nel modo giusto per ottenere il risultato.

Le prime missioni sembravano più facili, ma con il tempo ho visto crescere la complessità. La cosa che mi è piaciuta di più è stato il fatto che, per ogni errore che facevo, c'era sempre una soluzione che mi portava a riflettere meglio su ciò che stavo facendo. Non solo ho imparato ad usare il terminale in modo più consapevole, ma ho anche capito l'importanza di rimanere concentrato e non arrendersi troppo facilmente.

Inoltre, le missioni non erano solo tecniche, ma coinvolgevano anche un po' di "pensiero laterale" e logica, come nel caso dei messaggi criptati che ho dovuto decriptare. È stato davvero gratificante scoprire che, a volte, la soluzione era più semplice di quanto pensassi, ma richiedeva una comprensione precisa di come funzionano i comandi.

Alla fine, posso dire che GameShell non è stato solo un gioco, ma una vera e propria esperienza che mi ha permesso di imparare e migliorare in un ambiente divertente e stimolante. Mi ha dato anche la sensazione di progredire, di accumulare piccole vittorie con ogni missione completata.

Un'esperienza che mi ha fatto capire quanto possa essere potente il terminale, e quanto ci sia ancora da scoprire se ci si impegna veramente.

